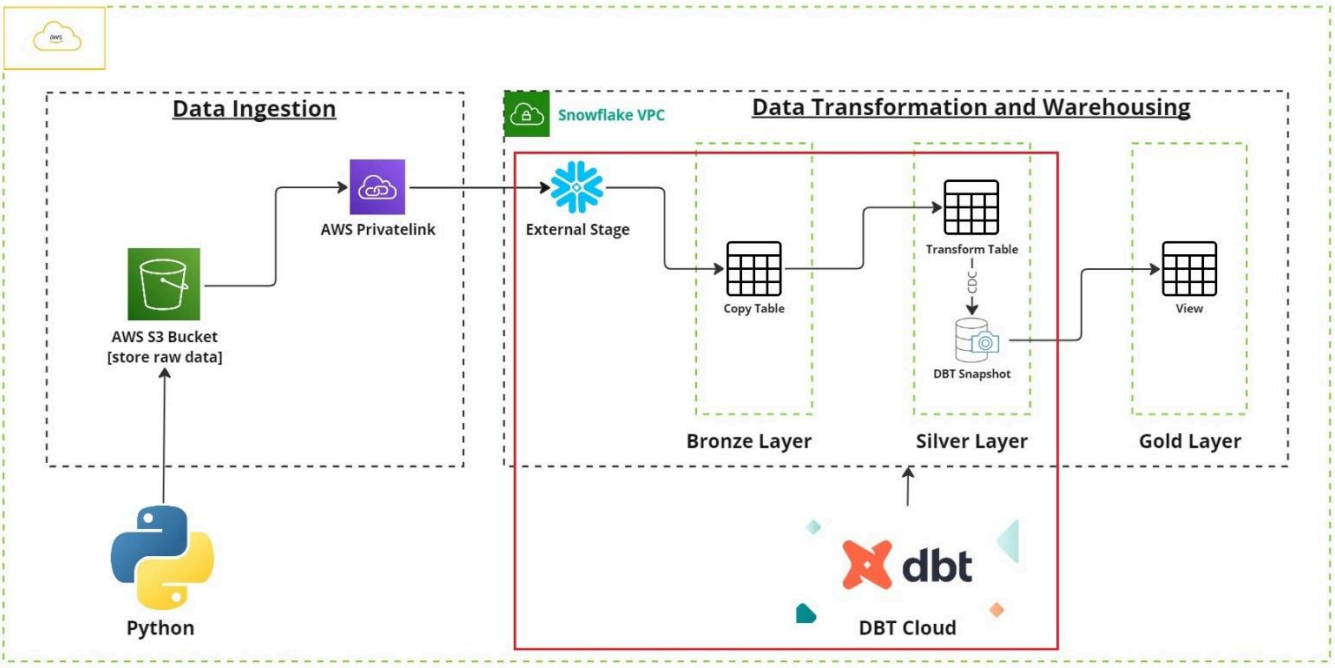
Technical Overview

In this demonstration, we are going to set up DBT along with Snowflake integration. Also, we will run sql logics using DBT to create our DWH layer with setup, integration with DynamoDB will also be done.

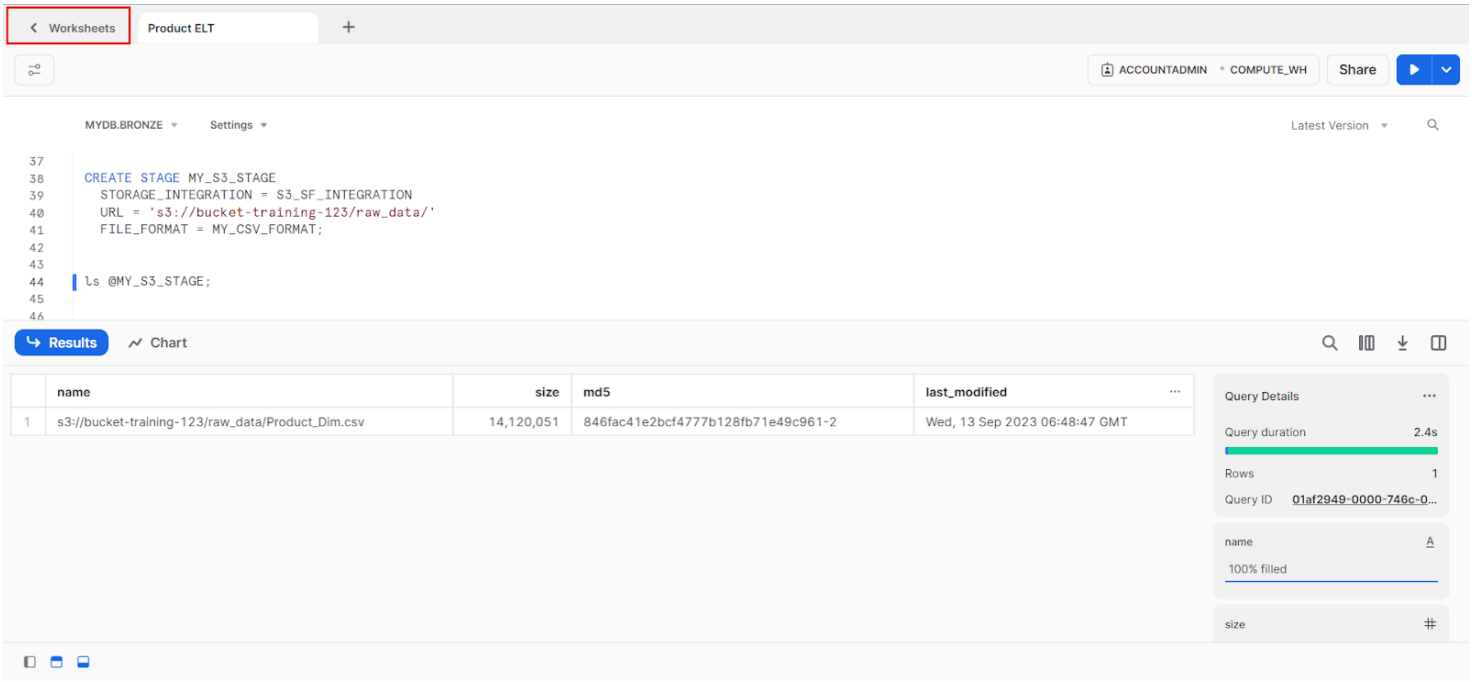


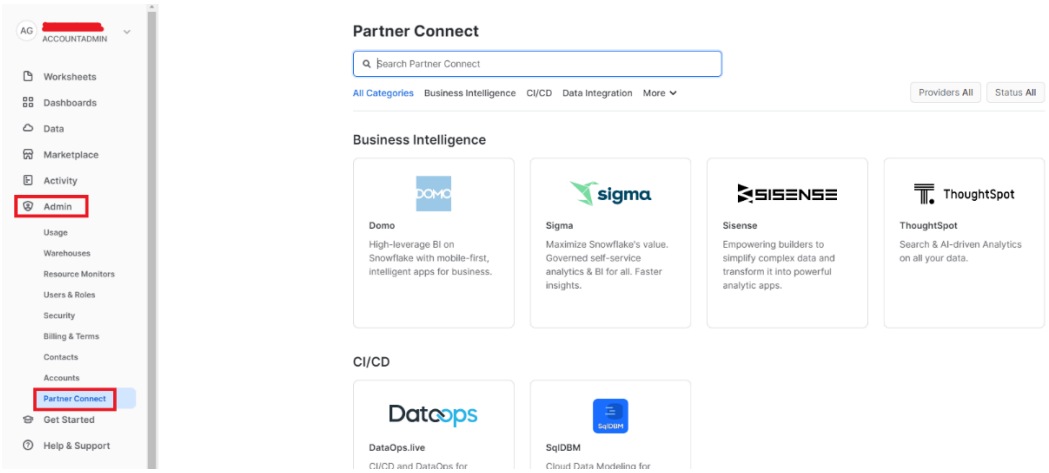
Pre-requisites:

- **Snowflake Account**

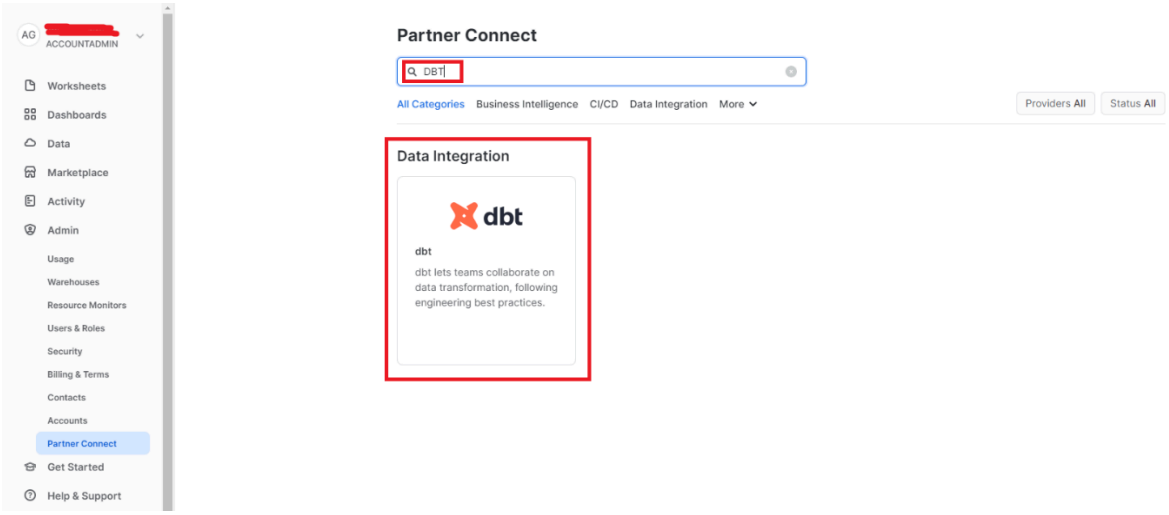
Create DBT Configuration

1. **In the Snowflake UI, click on the home icon on the top left side. To access Partner Connect, click on your Admin and then Partner Connect. Make sure you're still set in the ACCOUNTADMIN role.**

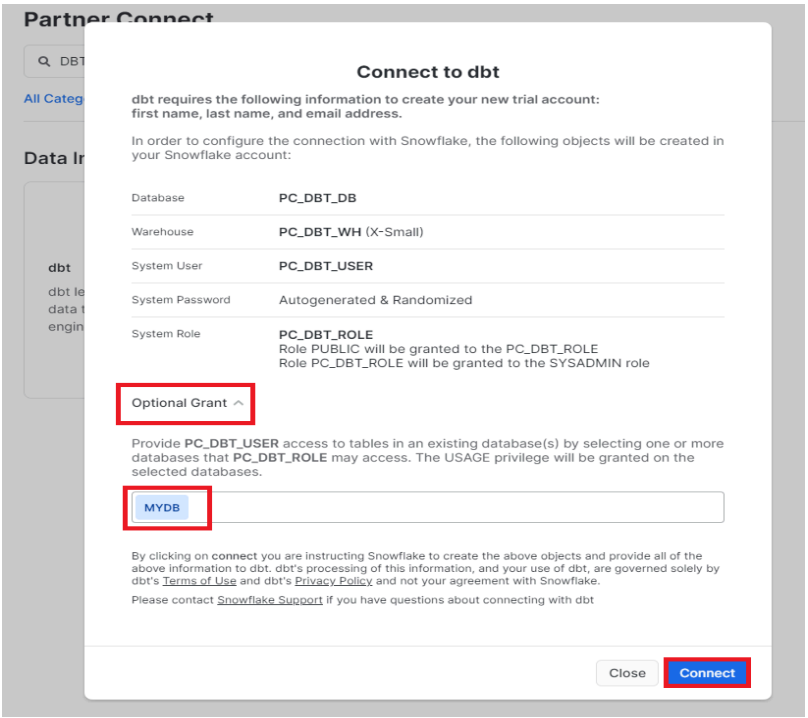




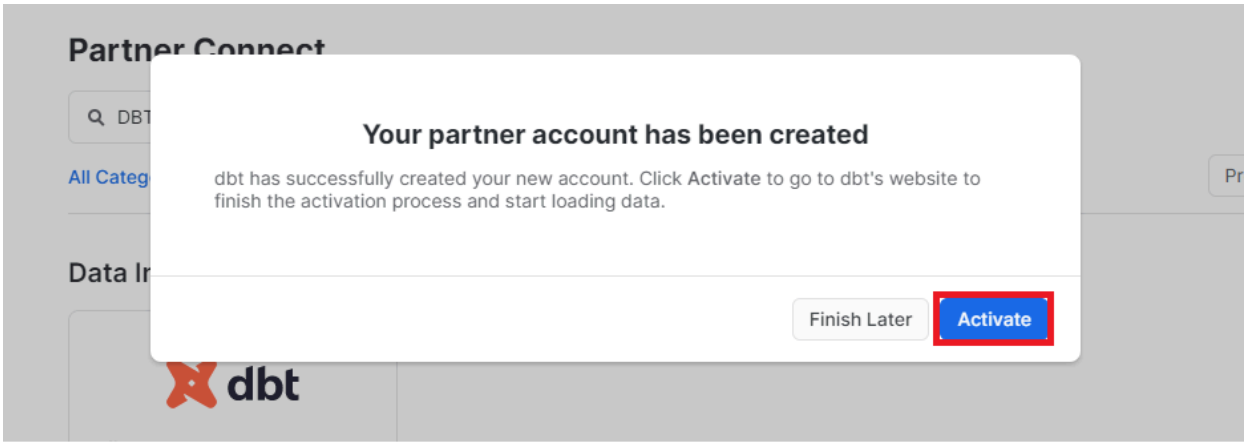
2. Now type DBT in the search bar and select it.



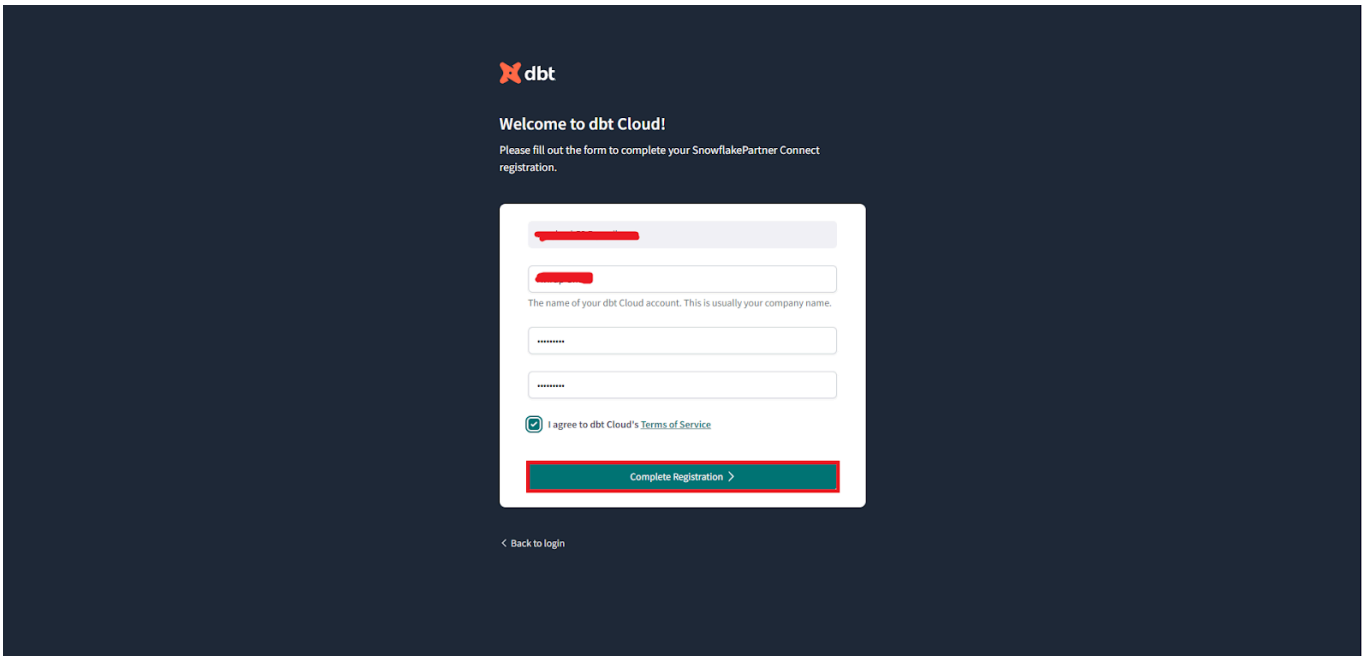
3. You should now see a popup that says Connect to DBT. Click on Optional Grant and you are going to add in the MYDB database. Click on Connect to proceed.



4. When the popup that says "Your partner account has been created", click on Activate.

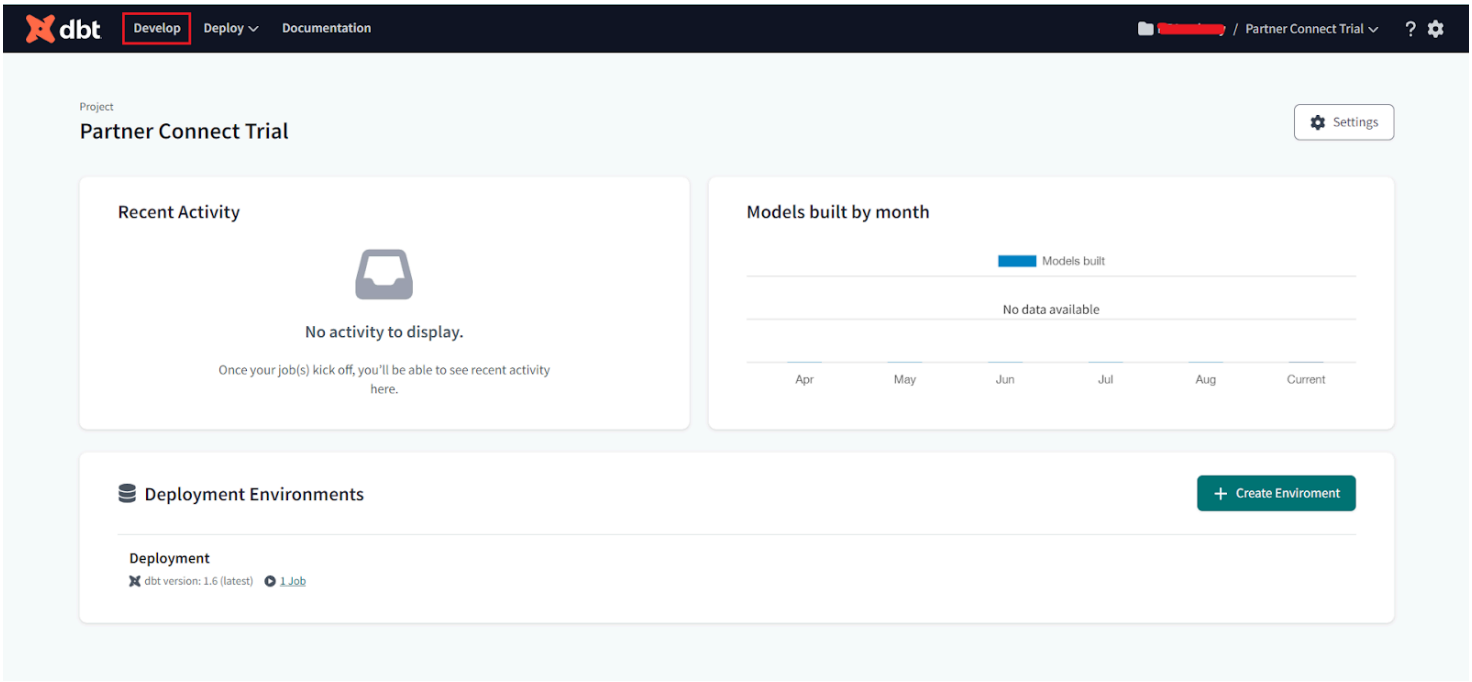


5. You should be redirected to a DBT Cloud registration page. Fill out the form. Make sure to save the password somewhere for login in the future. Click on Compete Registration.

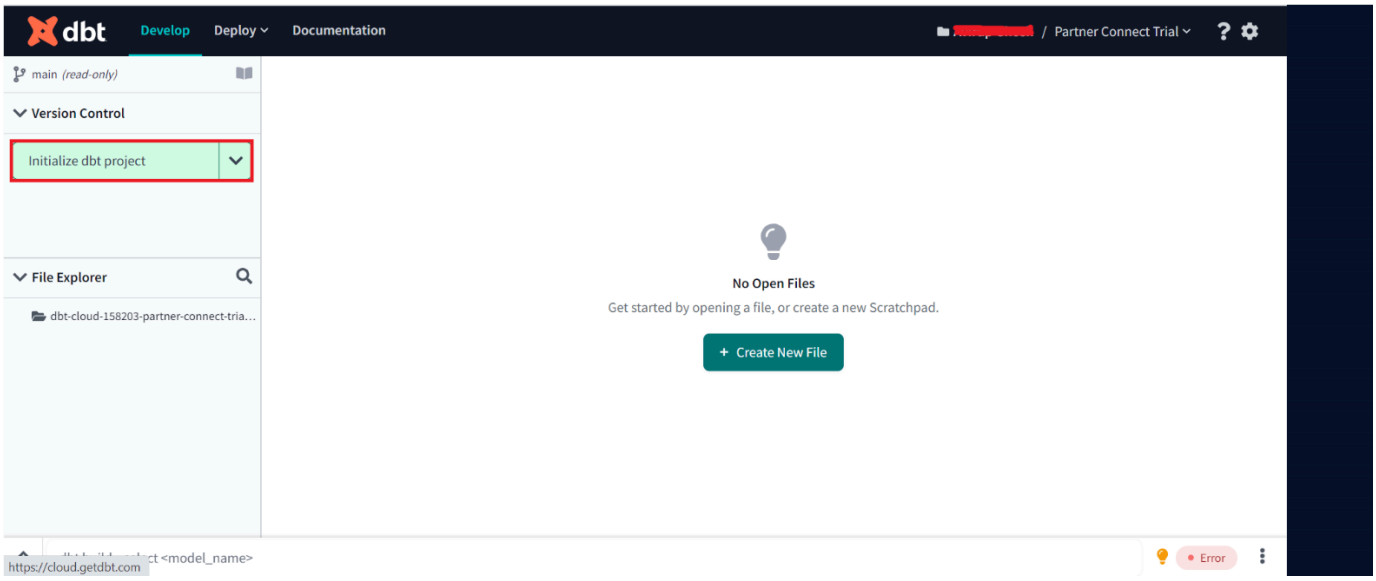


Setup Project Configuration

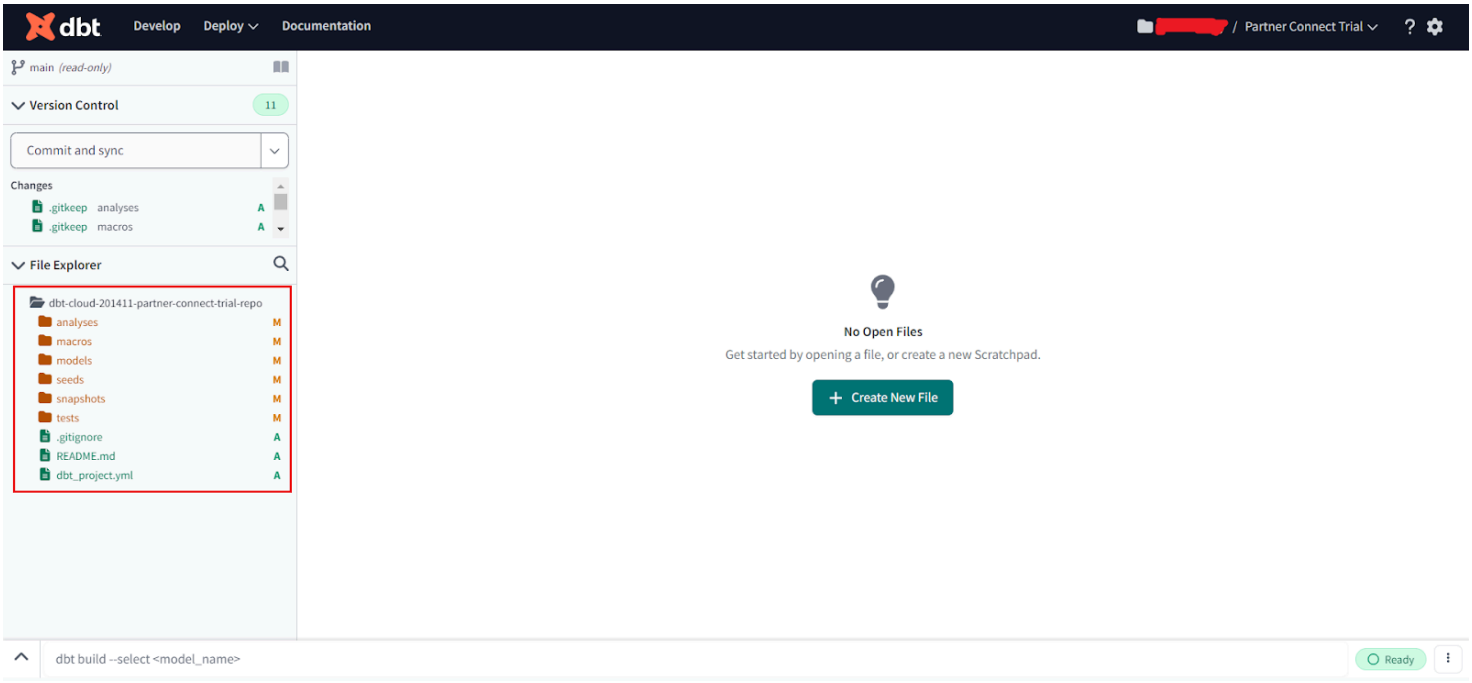
1. Now let's set up our DBT project. Click on Develop which is beside DBT icon on the top bar. This will spin up your IDE (Integrated Development Environment) where you will be developing your DBT Project.

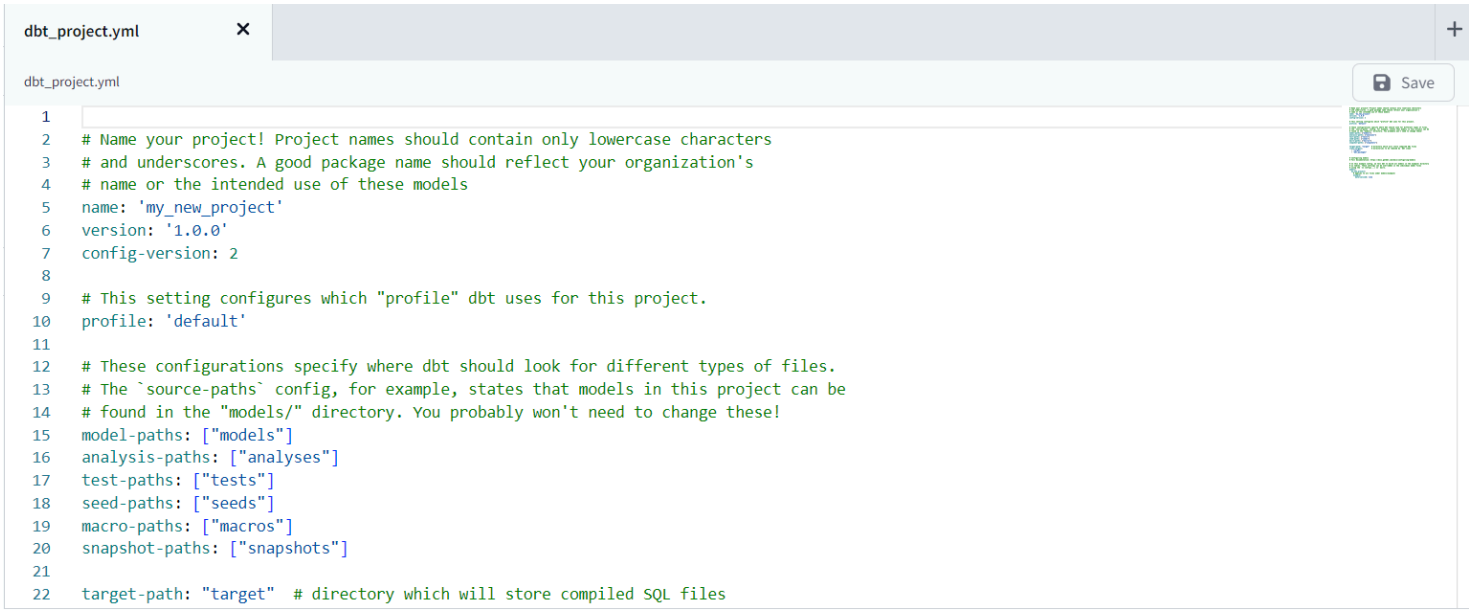


2. After the IDE loads, click Initialize your project to set up your DBT project. Once you click on it, DBT will generate our starter project with the core files and folders.

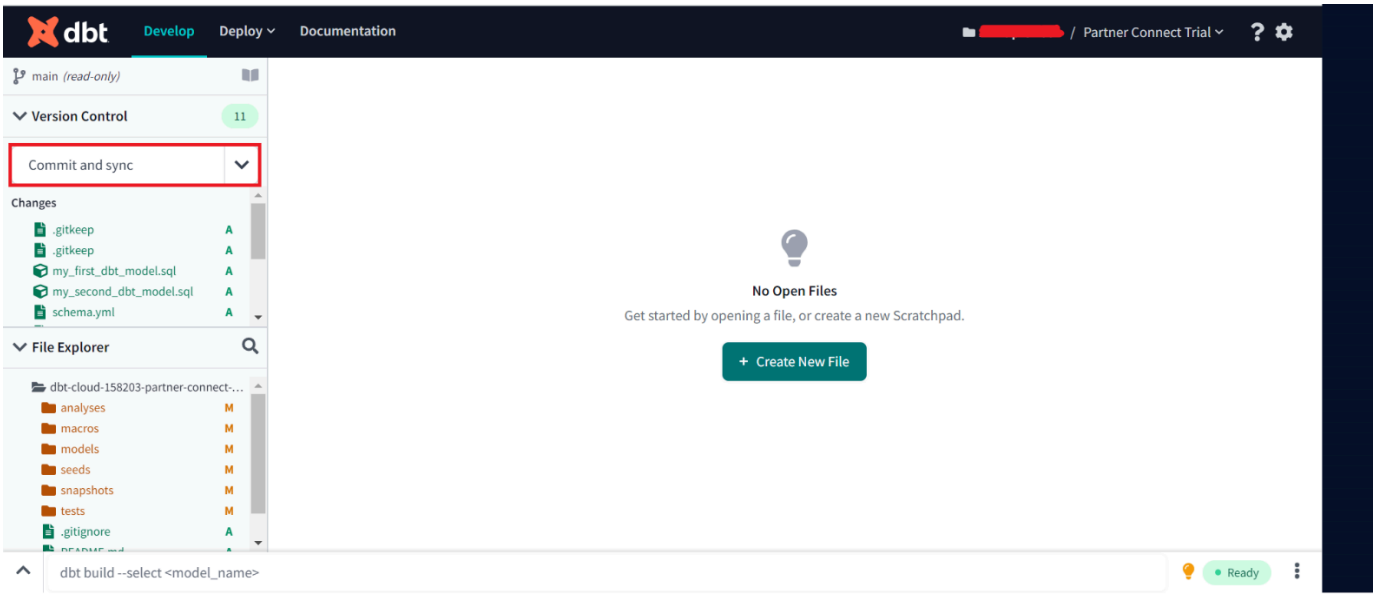


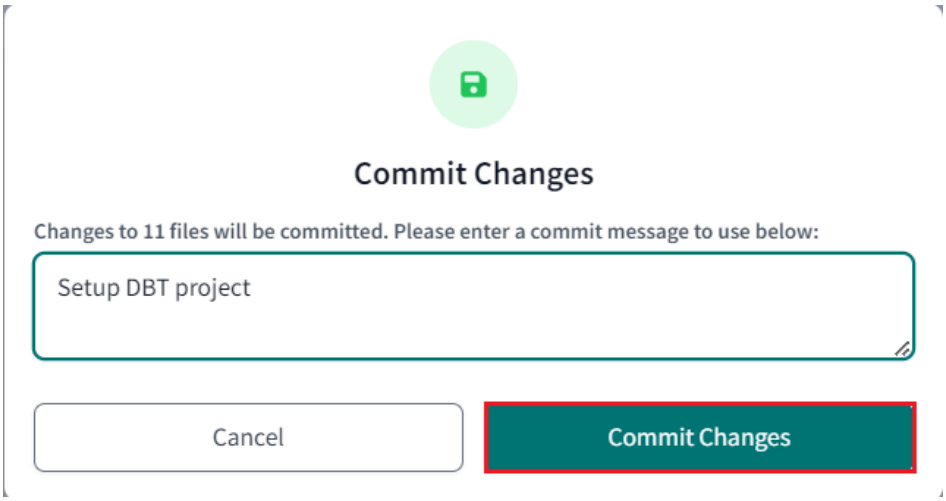
3. You should see new files and folders created in your file tree. These are all the files and folders you will need for a dbt project. Take a look around, especially at the dbt_project.yml and what's in the model's directory.



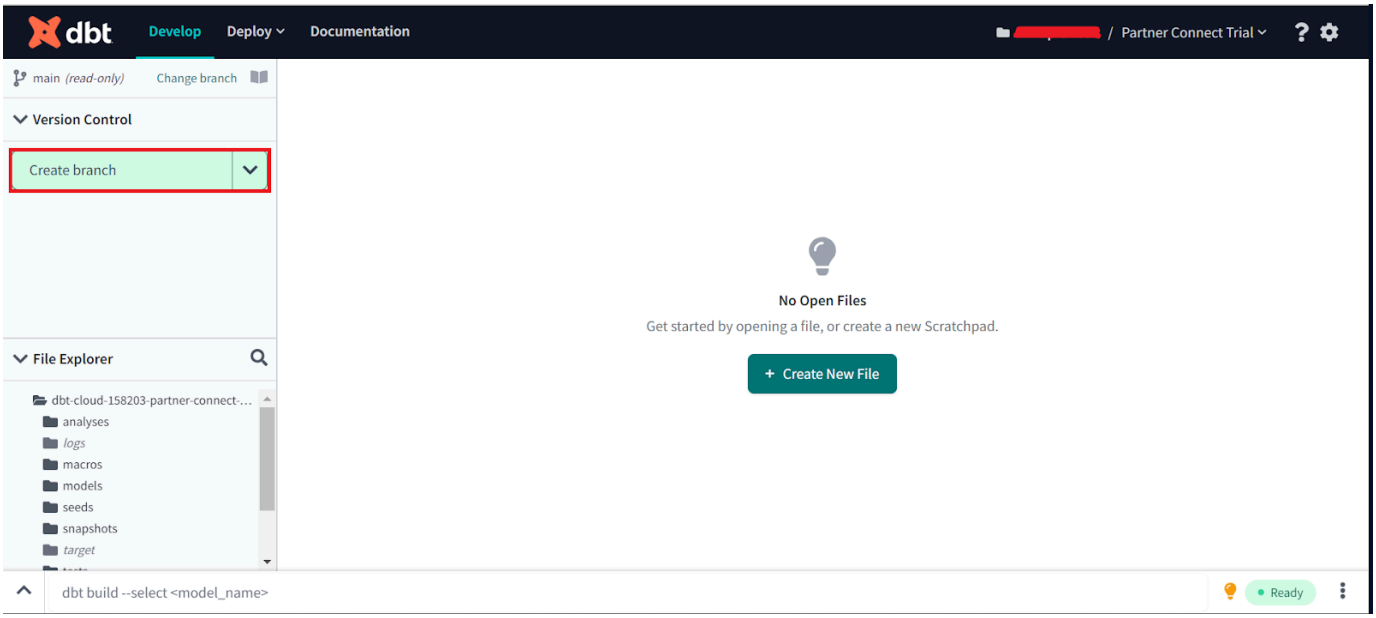


4. Click Commit and sync. Enter a commit message, and then click Commit again to commit your work to your master branch.

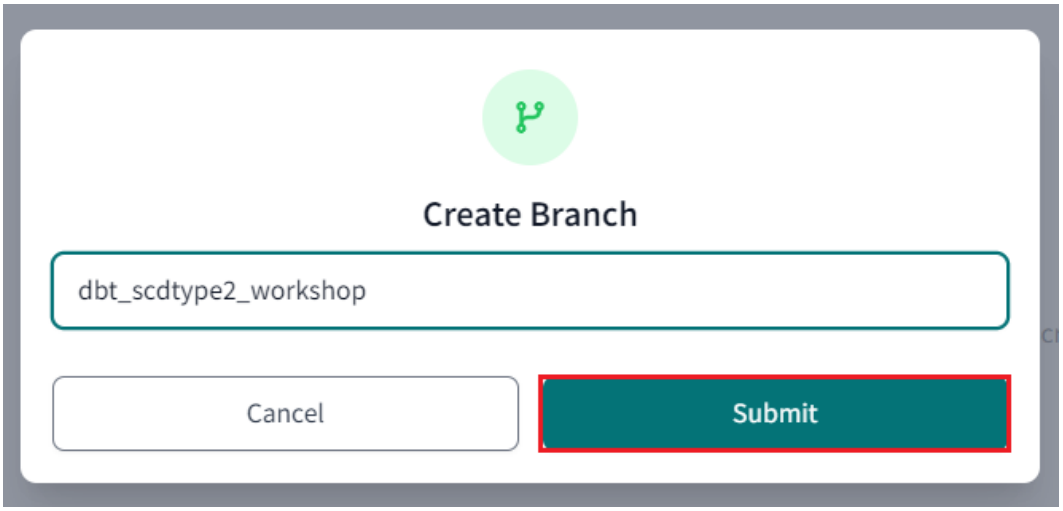




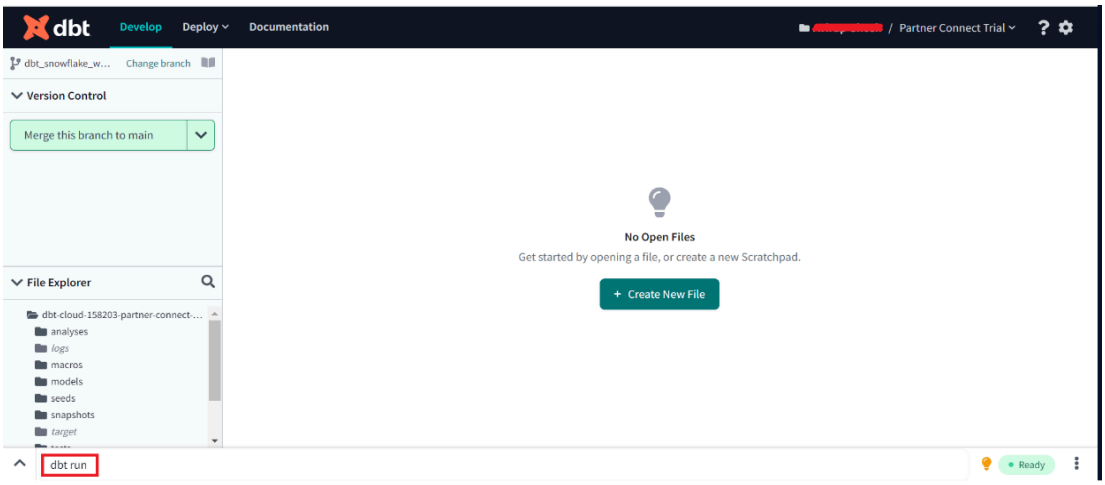
5. Click Create branch to check out a new git branch to start developing.



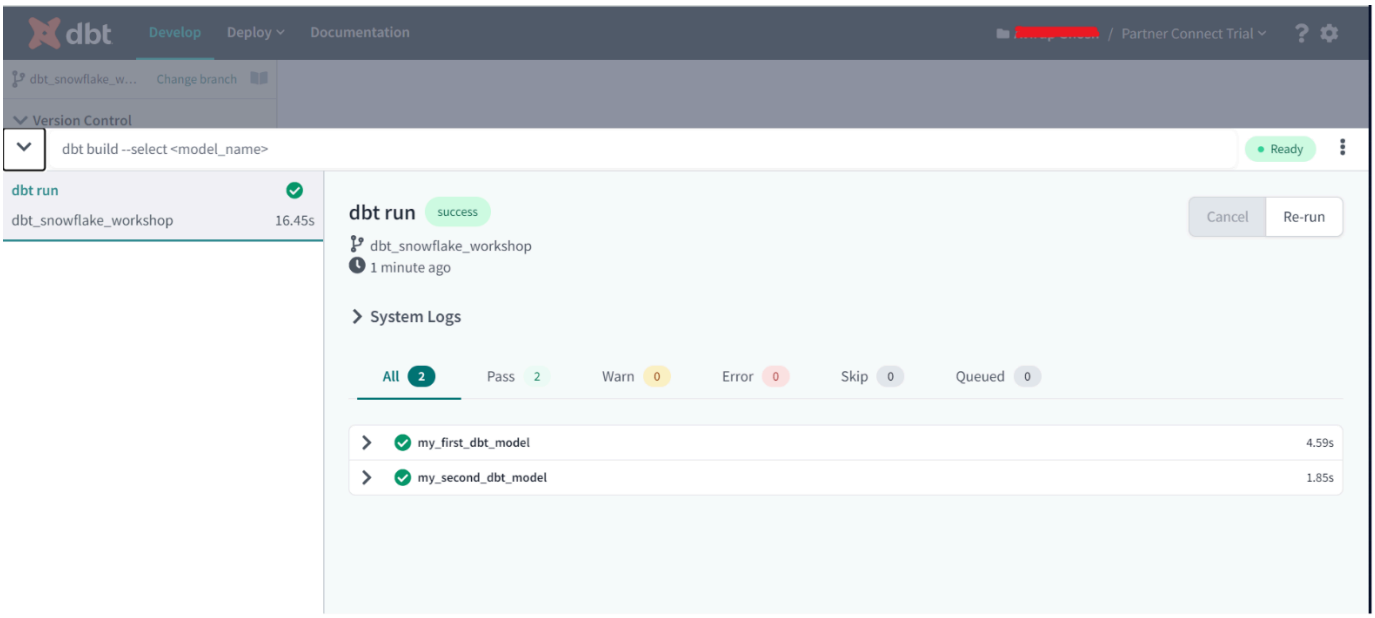
6. Name the branch dbt_scdtype2_workshop and click Submit.



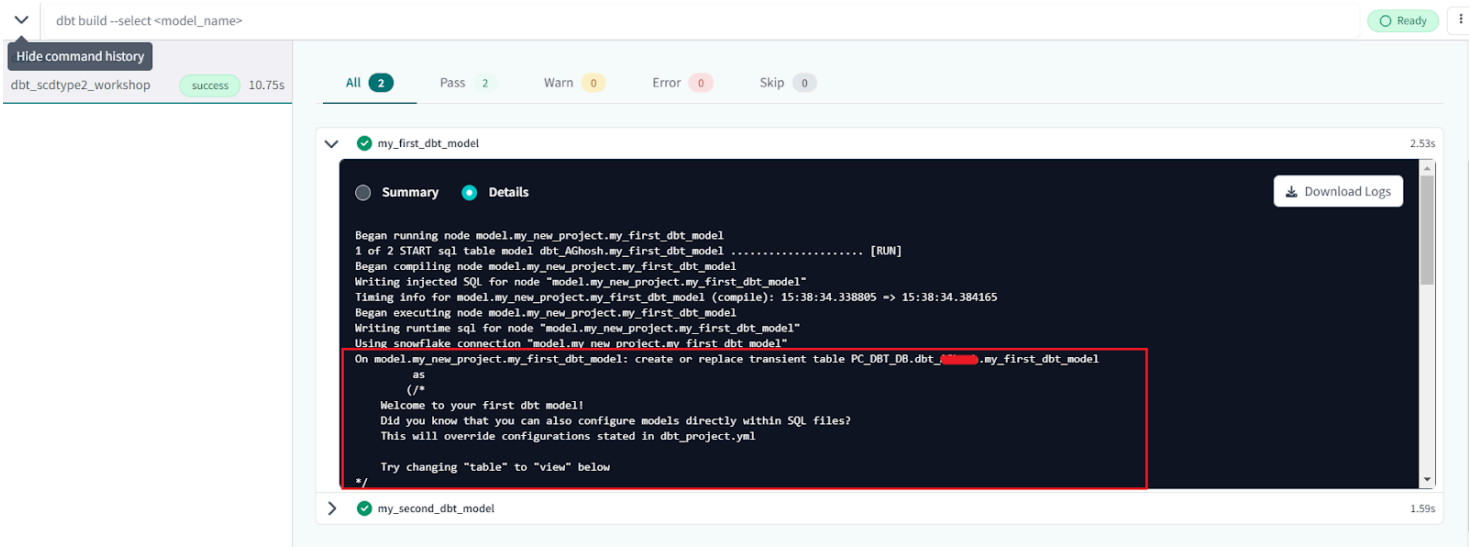
7. Now let's validate that everything was initialized correctly by running the sample models that come with the starter DBT project. Type in dbt run on the command line at the bottom and hit Enter.



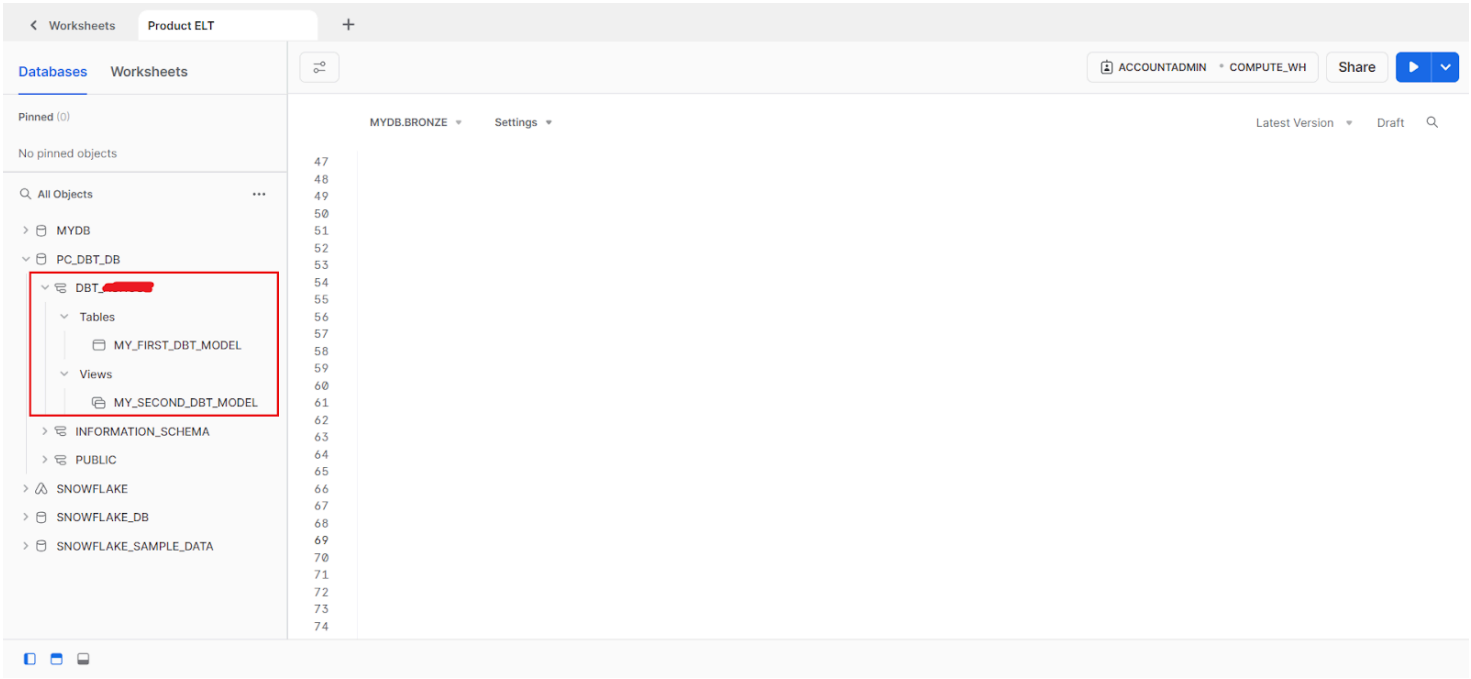
8. The output should look like below, confirming DBT was able to connect to Snowflake and successfully execute the sample models in the model's folder:



9. If you want to see what is executed against Snowflake, click into one of the model tabs and click on details.

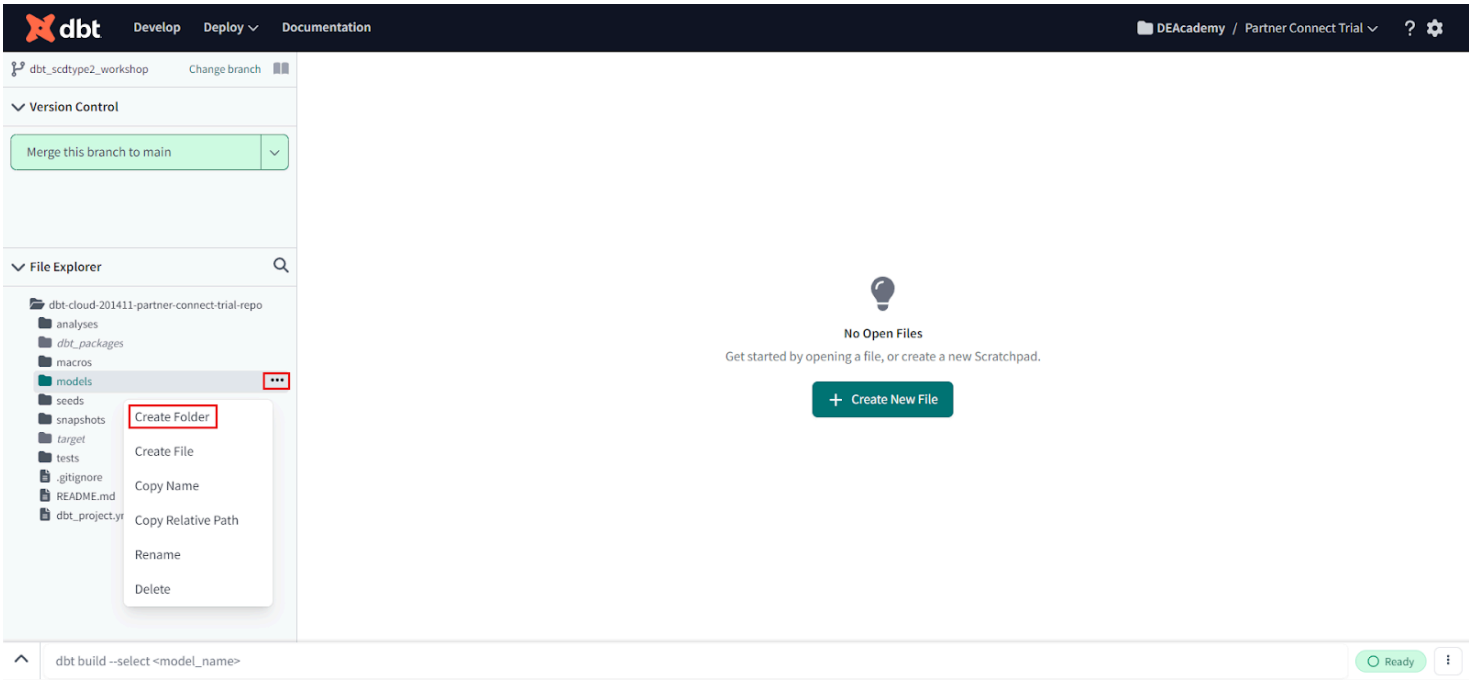


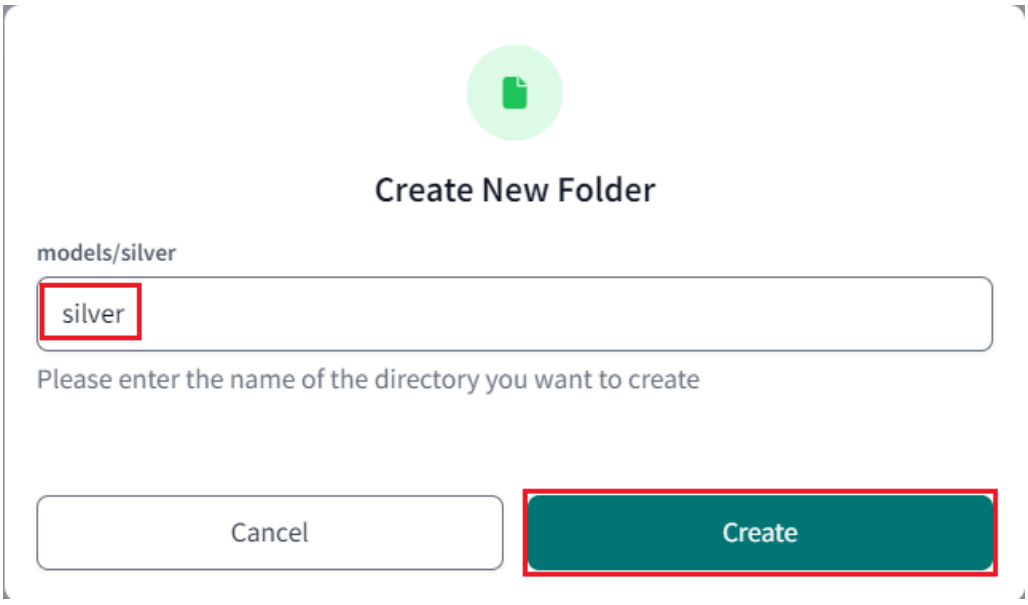
10. Now let's visually confirm the objects in Snowflake. Switch to the Snowflake UI (classic console) and refresh database objects. Expand the database PC_DBT_WH. Then the development schema with your first initial and last name. And lastly Tables and Views. You should see the table MY_FIRST_DBT_MODEL and the view MY_SECOND_DBT_MODEL.



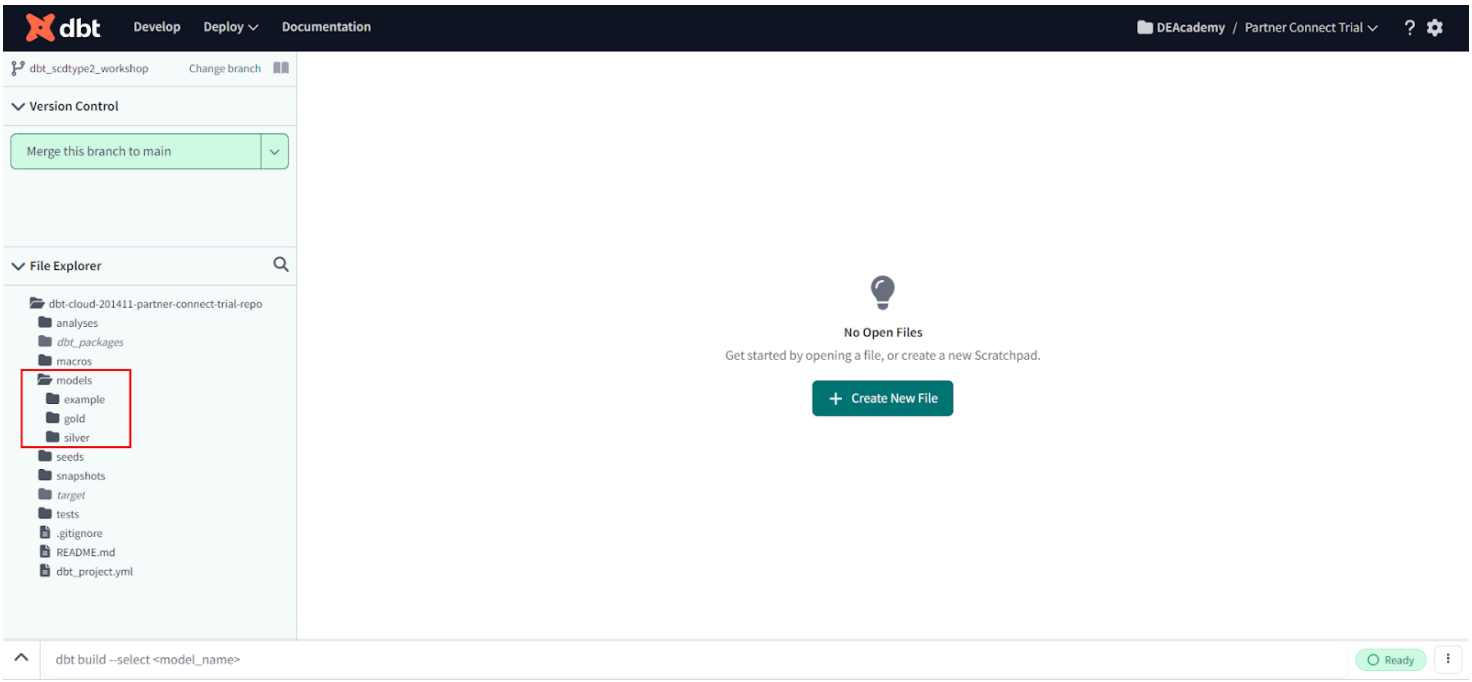
We are going to start by adding a few more things to our dbt project configuration in order to improve maintainability.

11. To add new folders, go over the model's directory in the file explorer tree and click on the '...' on the right next to the model's directory. Click on Create Folder and type into it silver. Click Create after you finish typing and you should see a new folder in the file tree.

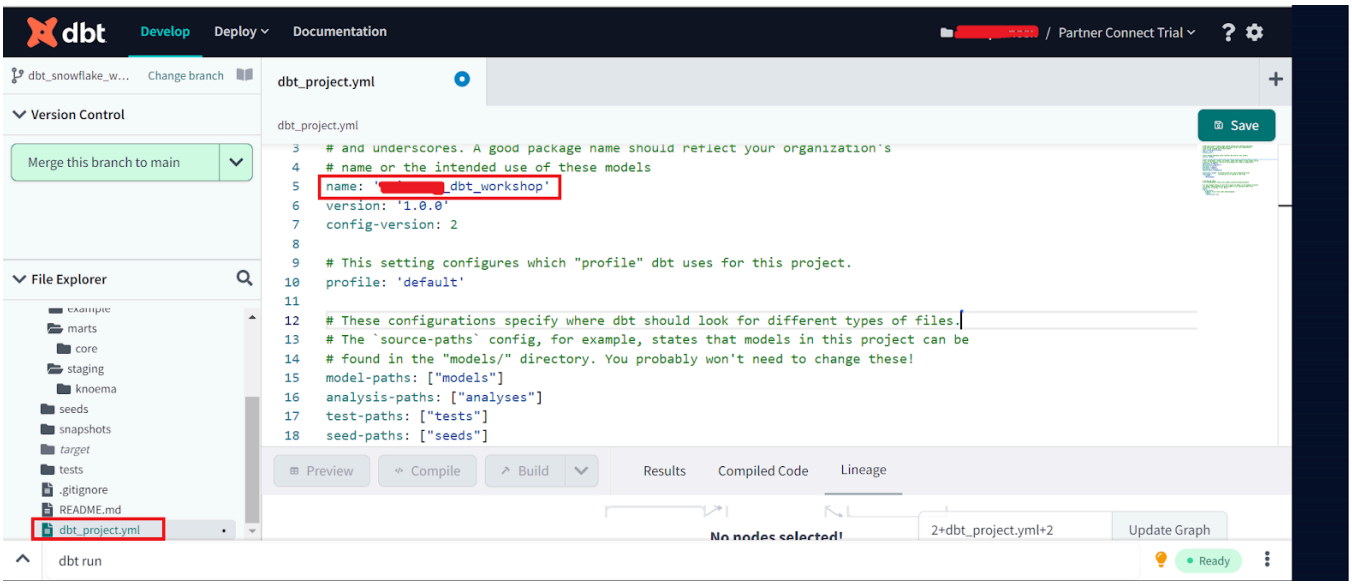


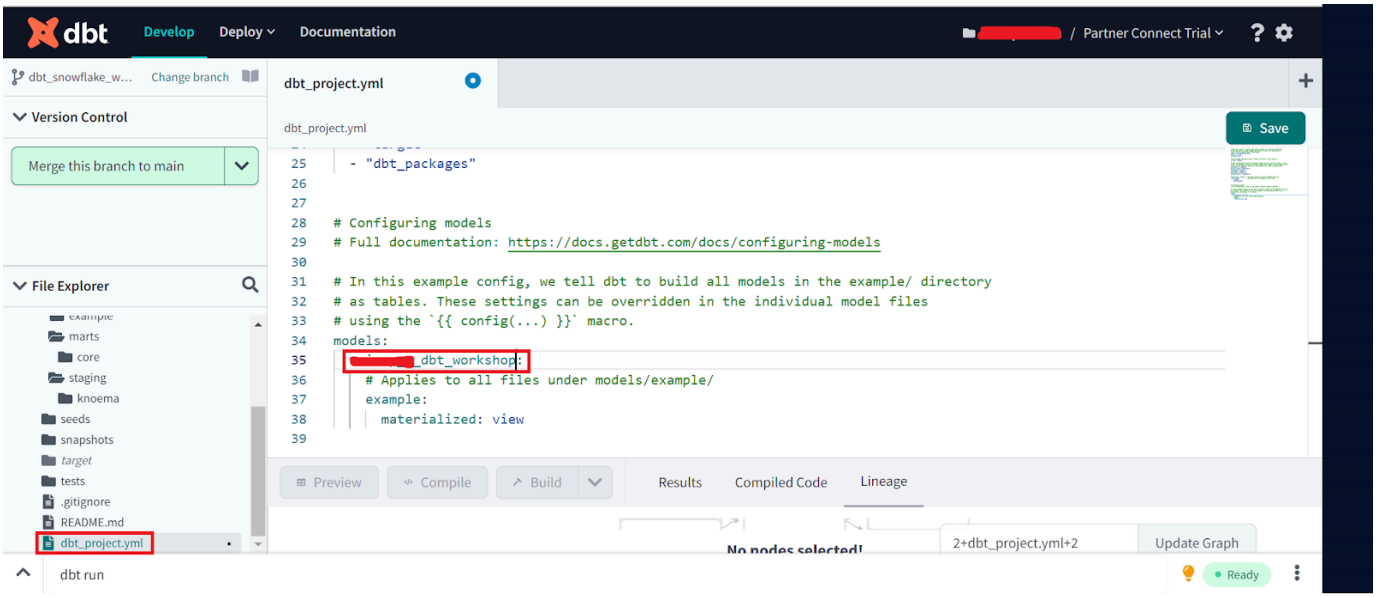


12. Click on the '...' next to models again. This time, we are going to fill the file path as gold. This will create a new folder under models called gold.



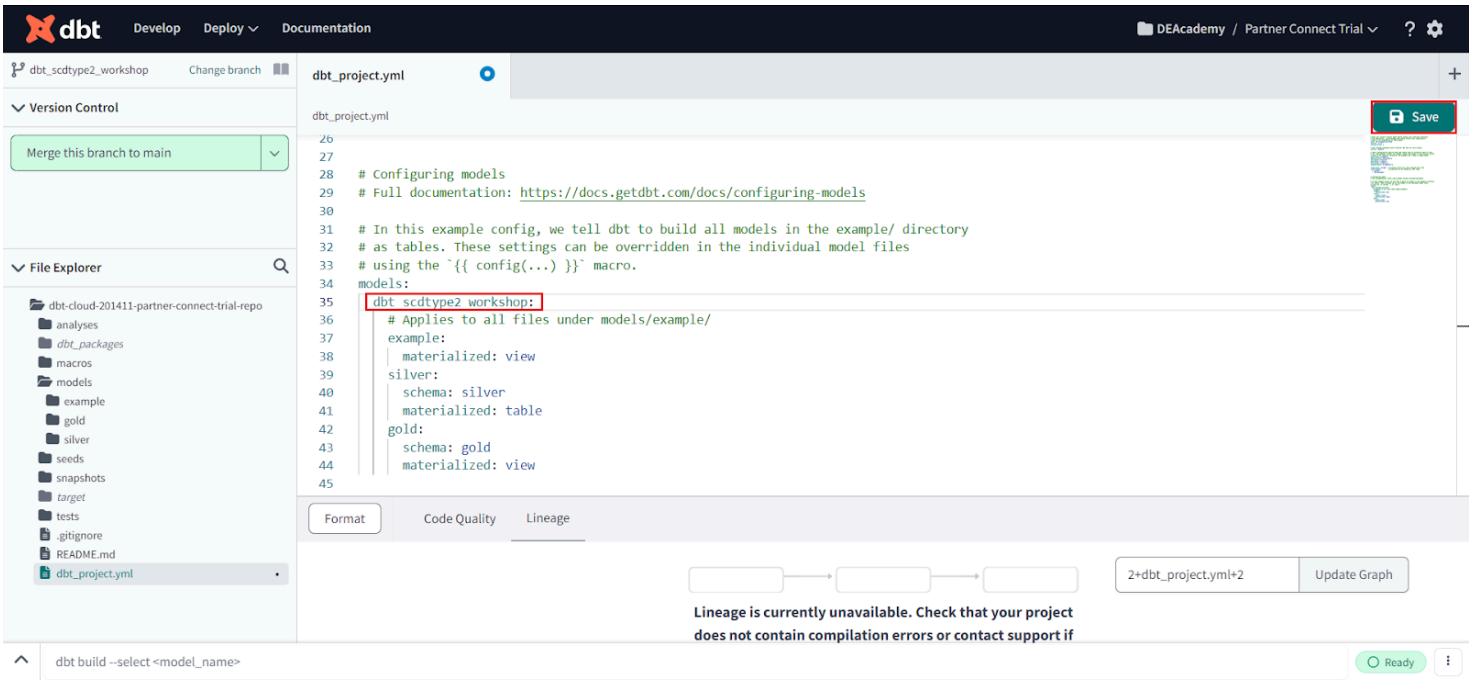
13. Next, let's update the dbt_project.yml file. Click on the file dbt_project.yml. Update the project name from "my_new_project" to "<your_name>_dbt_workshop" at line 5.





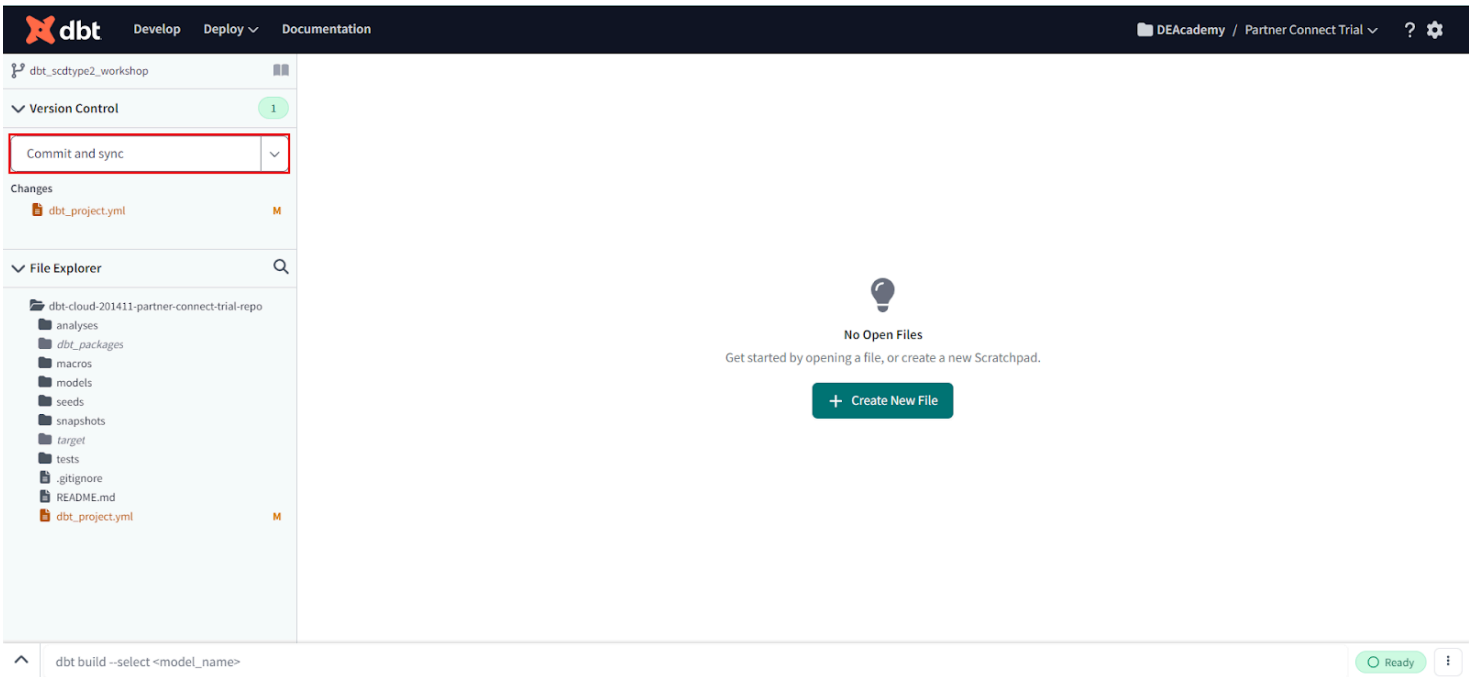
14. Copy code below and paste into line 34 to line 38. Remember to replace <your_name>.

```
models:
  <your_name>_dbt_workshop:
      example:
          schema: example
      silver:
          schema: silver
          materialized: table
      gold:
          schema: gold
          materialized: view
```

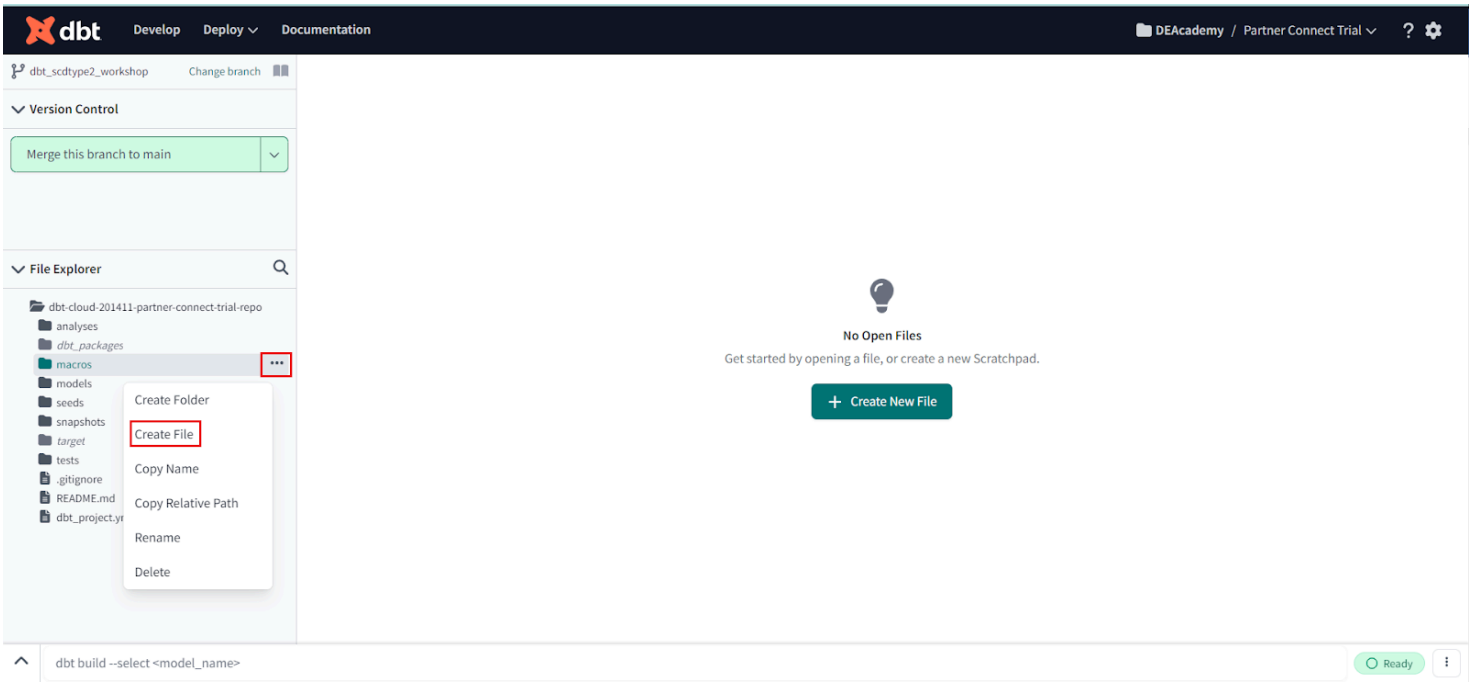


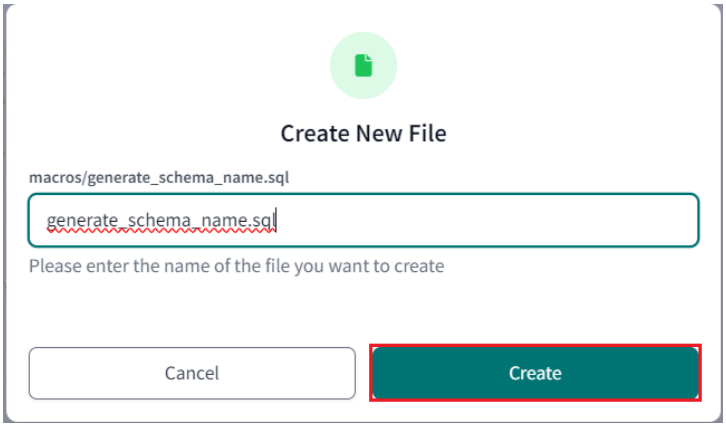
Click on Save to save the file.

15. Click on "Commit and sync" and add message. Click on Commit changes.



16. Create a new file in the macros folder named generate_schema_name.sql. Click on Create to proceed.





17. Copy and paste the below code into file generate_schema_name.sql to generate the macro. Click on Save.

```
{% macro generate_schema_name(custom_schema_name, node) -%}
```

```
    {%- set default_schema = target.schema -%}

    {%- if custom_schema_name is none -%}

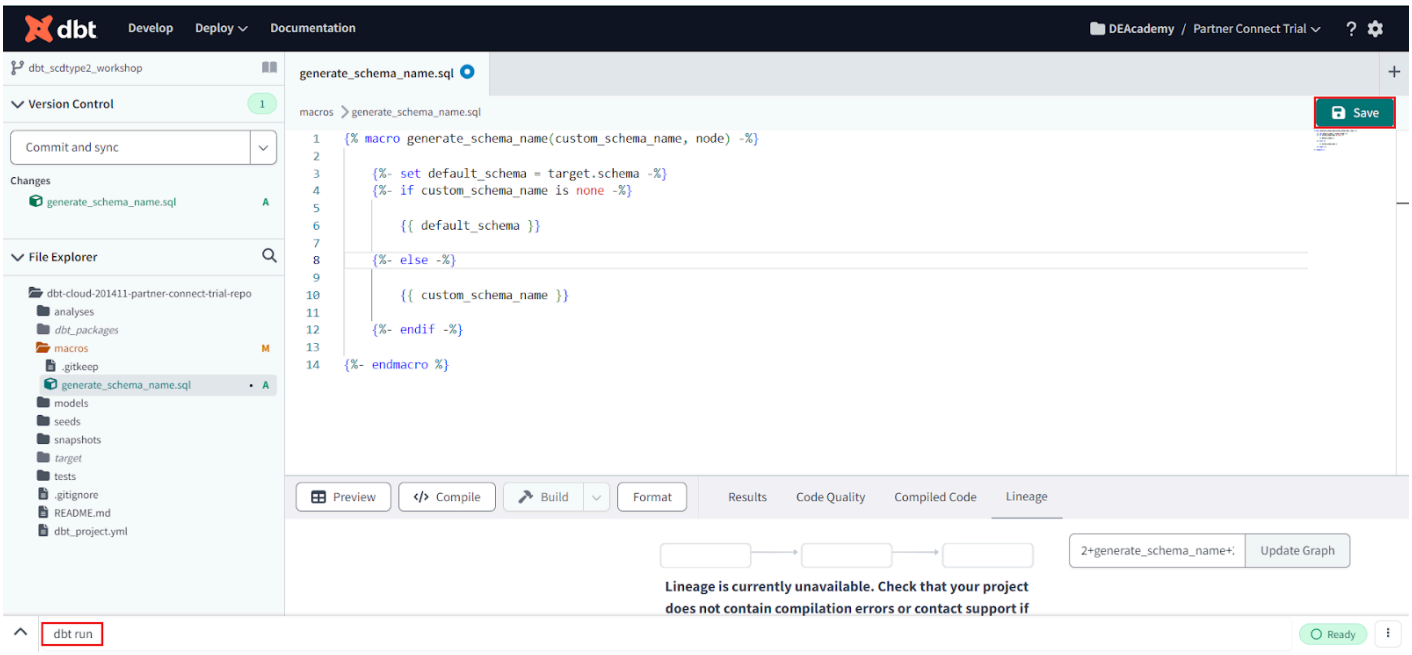
        {{ default_schema }}

    {%- else -%}

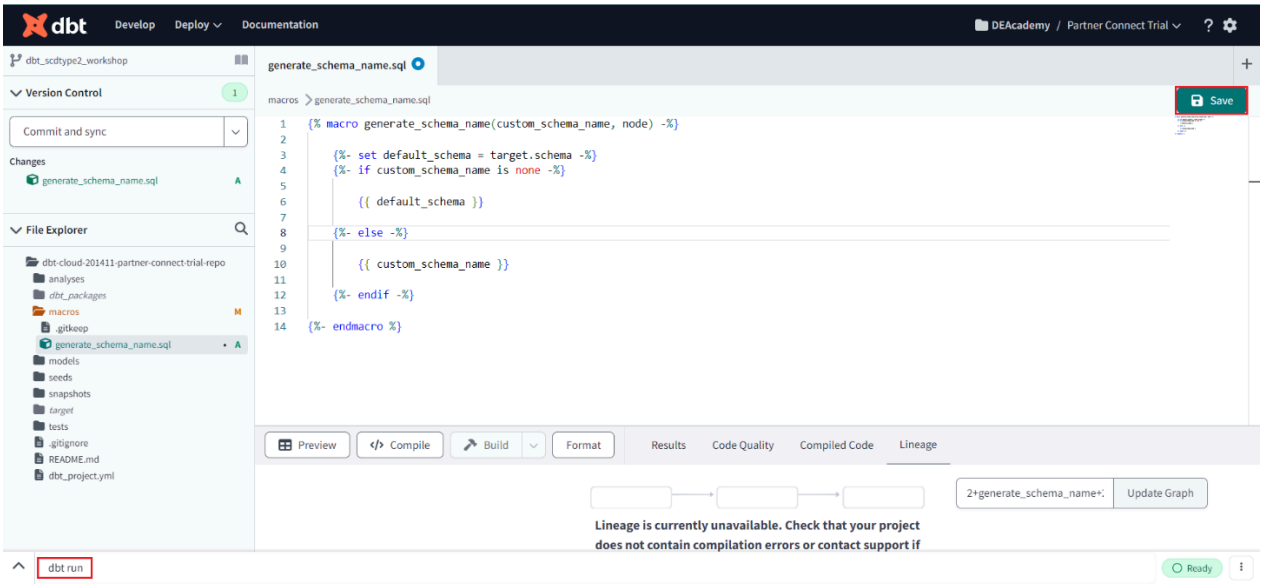
        {{ custom_schema_name }}

    {%- endif -%}

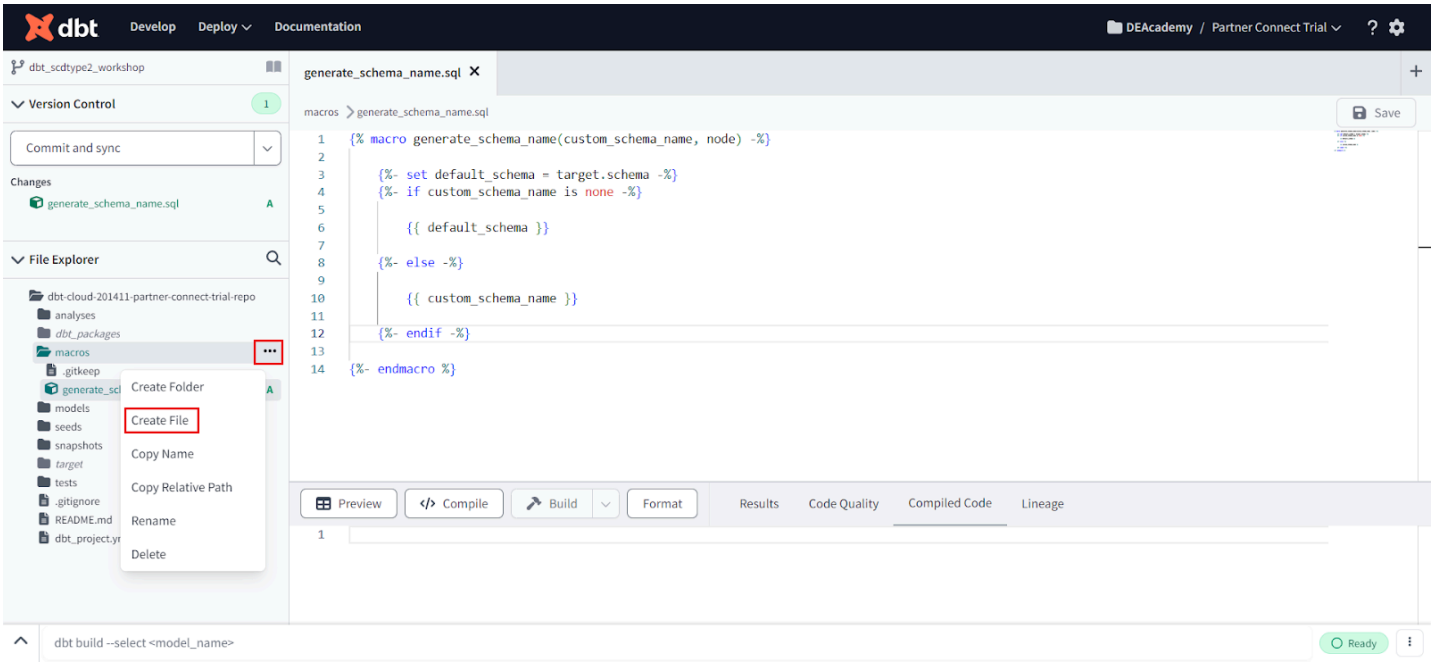
{%- endmacro %}
```

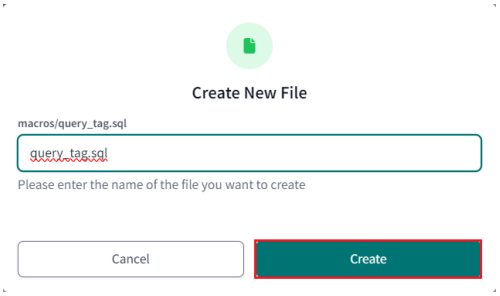


18. Do another dbt run on the command line. Hit Enter on your keyboard to run.



19. Create a file in the "macros" folder called query_tag.sql. Copy and paste the following code. Click on Save.





```
{% macro set_query_tag() -%}

  {% set new_query_tag = model.name %} {# always use model name #}

  {% if new_query_tag %}

    {% set original_query_tag = get_current_query_tag() %}

    {{ log("Setting query_tag to '" ~ new_query_tag ~ "'. Will reset to '" ~ original_query_tag ~ "' after materialization.") }}

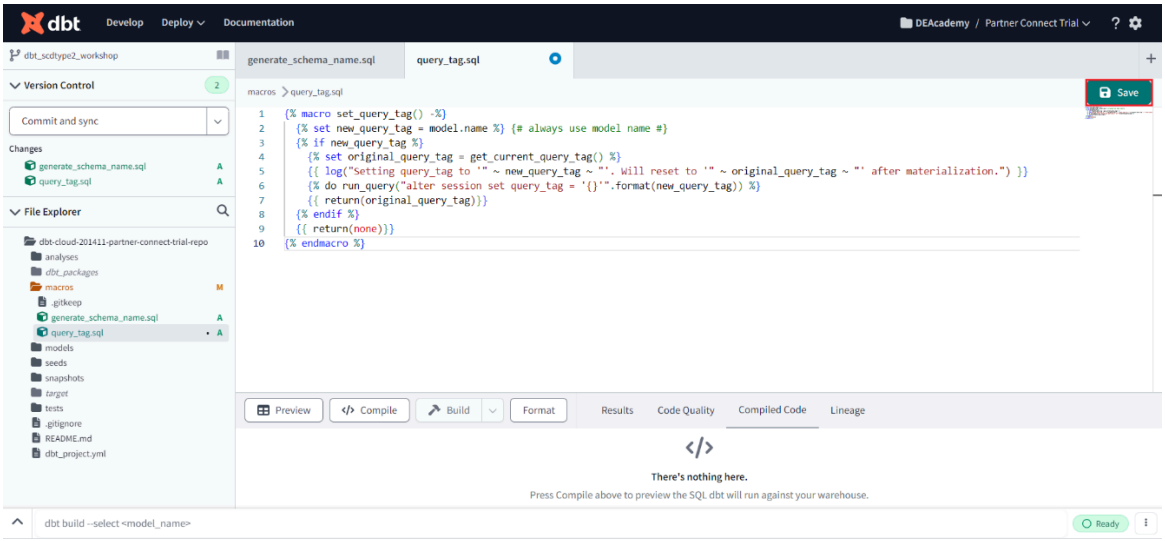
    {% do run_query("alter session set query_tag = '{}'".format(new_query_tag)) %}

    {{ return(original_query_tag)}}

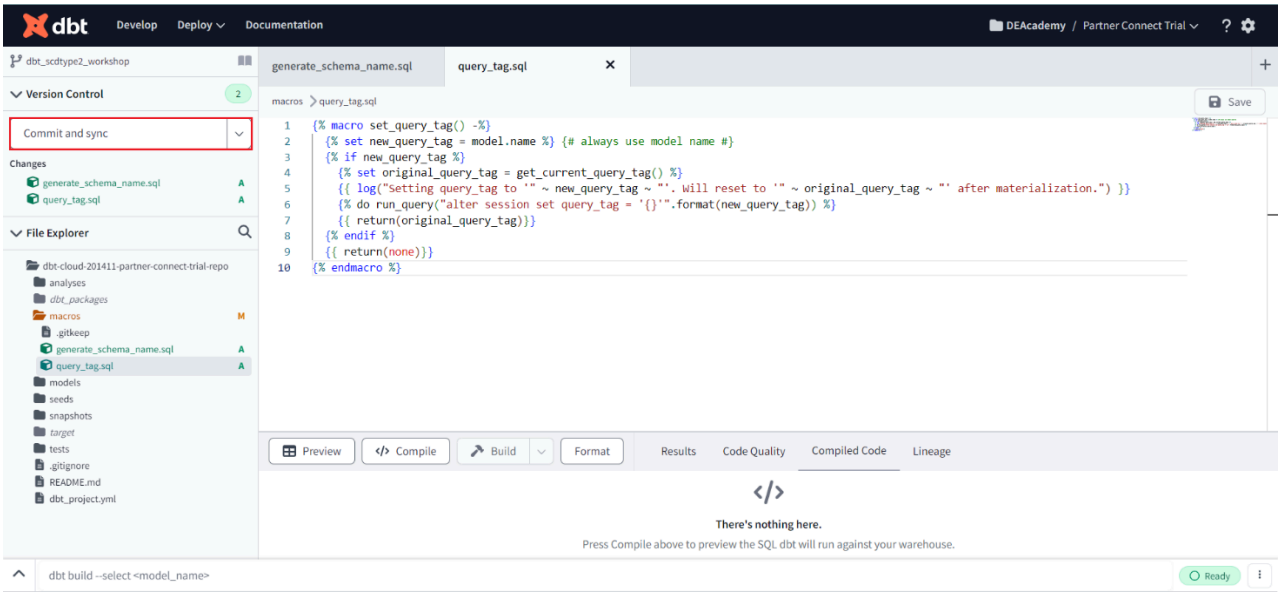
  {% endif %}

  {{ return(none)}}

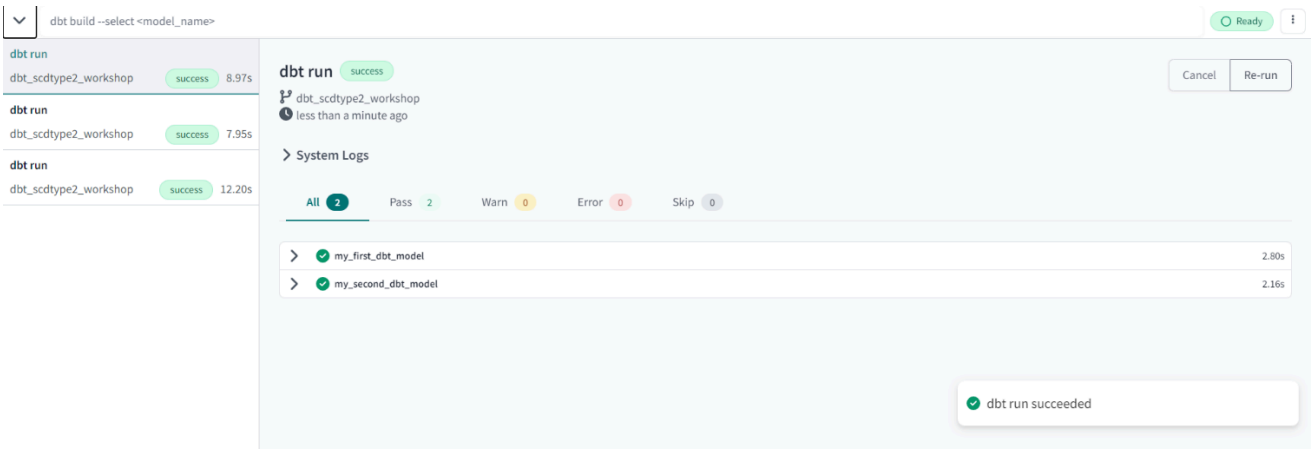
{% endmacro %}
```



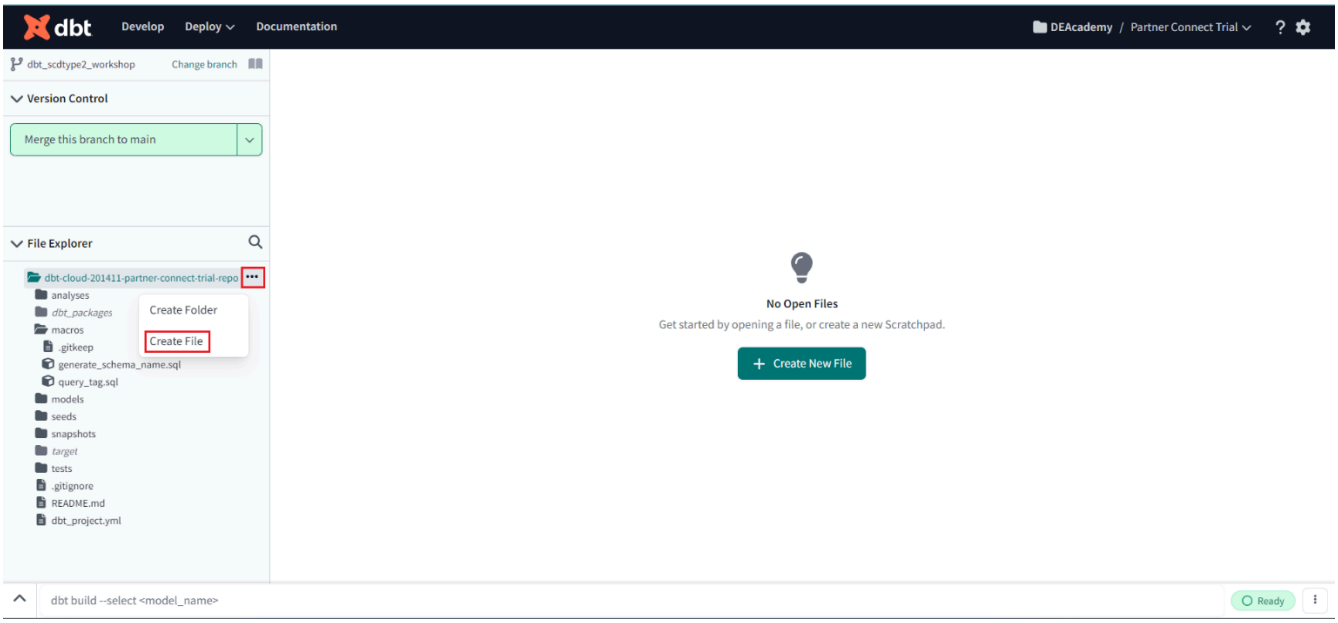
20. Click on Commit and sync. Add message and click on Commit.



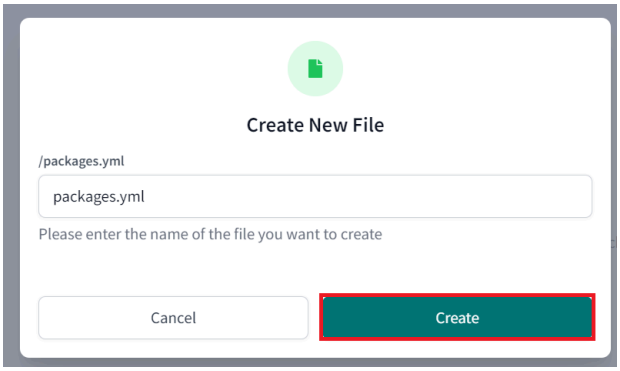
21. Now, do a dbt run again on the command line. Hit Enter on your keyboard to run.



22. Now we are going to install a dbt package. Create a file called packages.yml at the same level as your dbt_project.yml file.

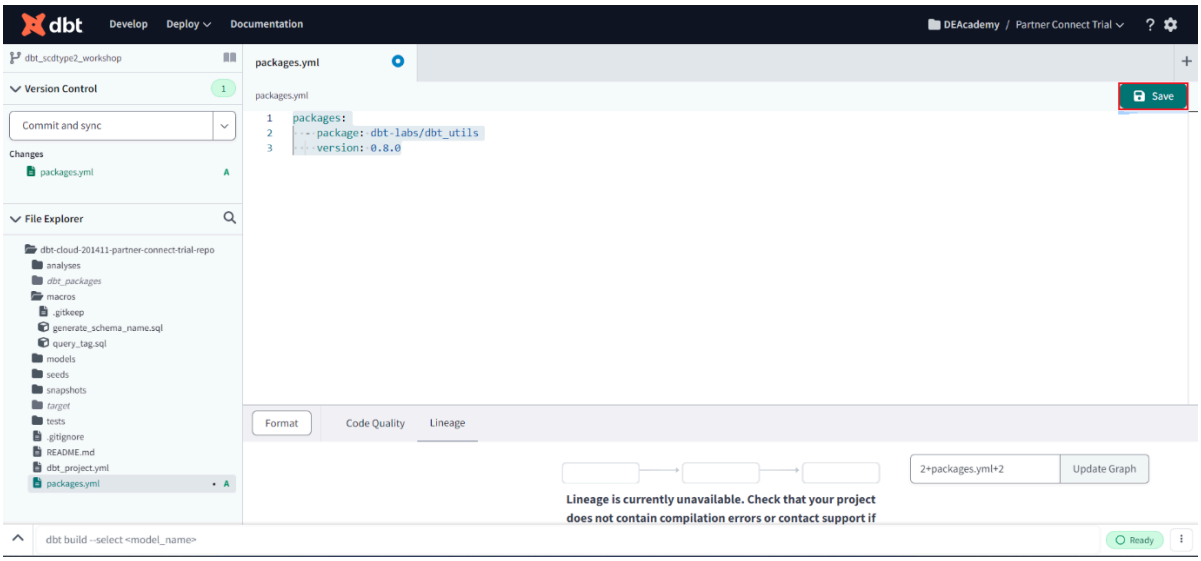


23. Add the name packages.yml and click on Create.

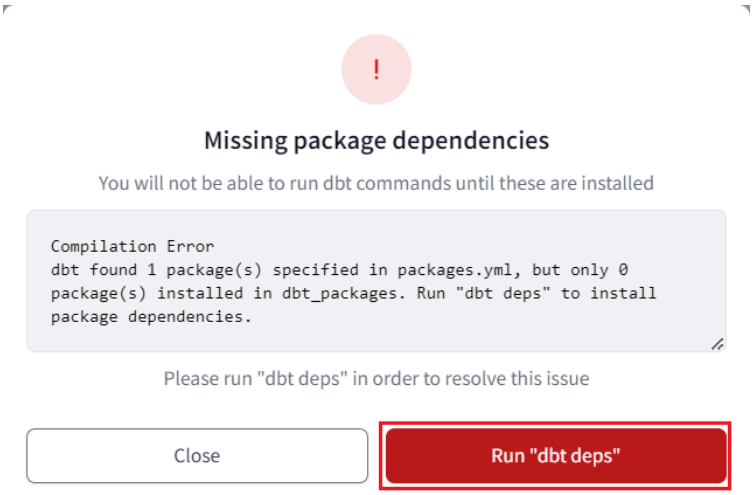


24. Copy and paste the following code into file packages.yml. Click on Save.

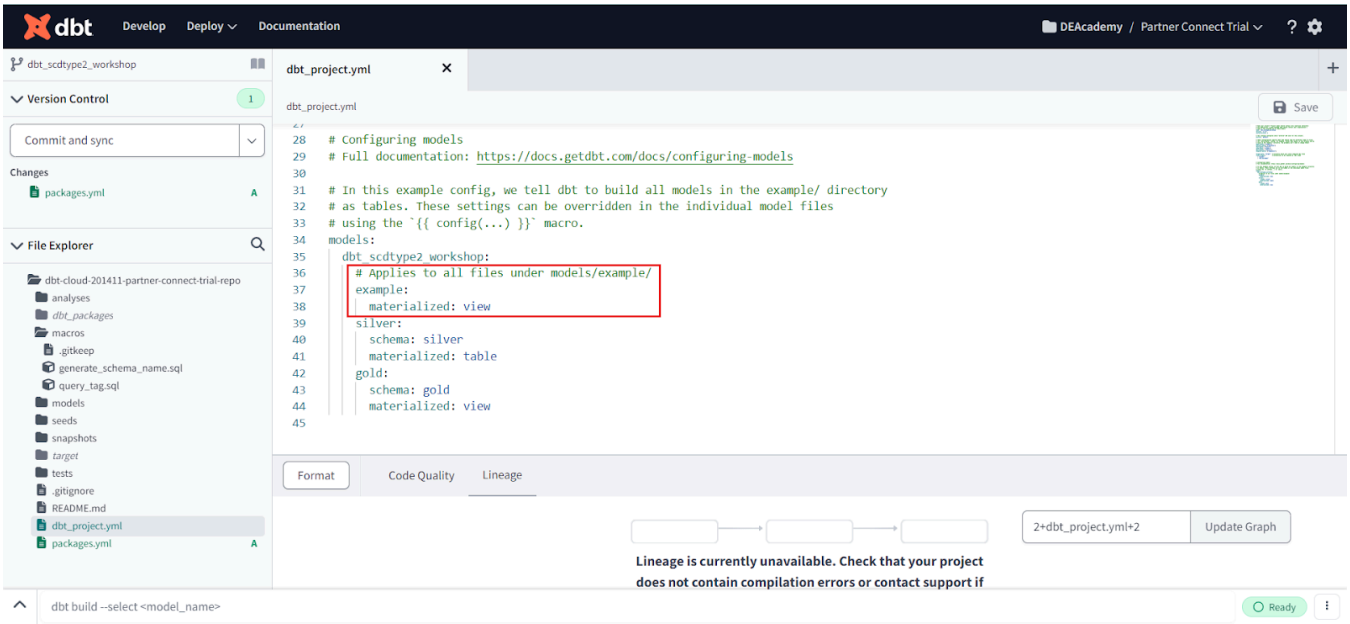
```
packages:
  - package: dbt-labs/dbt_utils
    version: 0.8.0
```



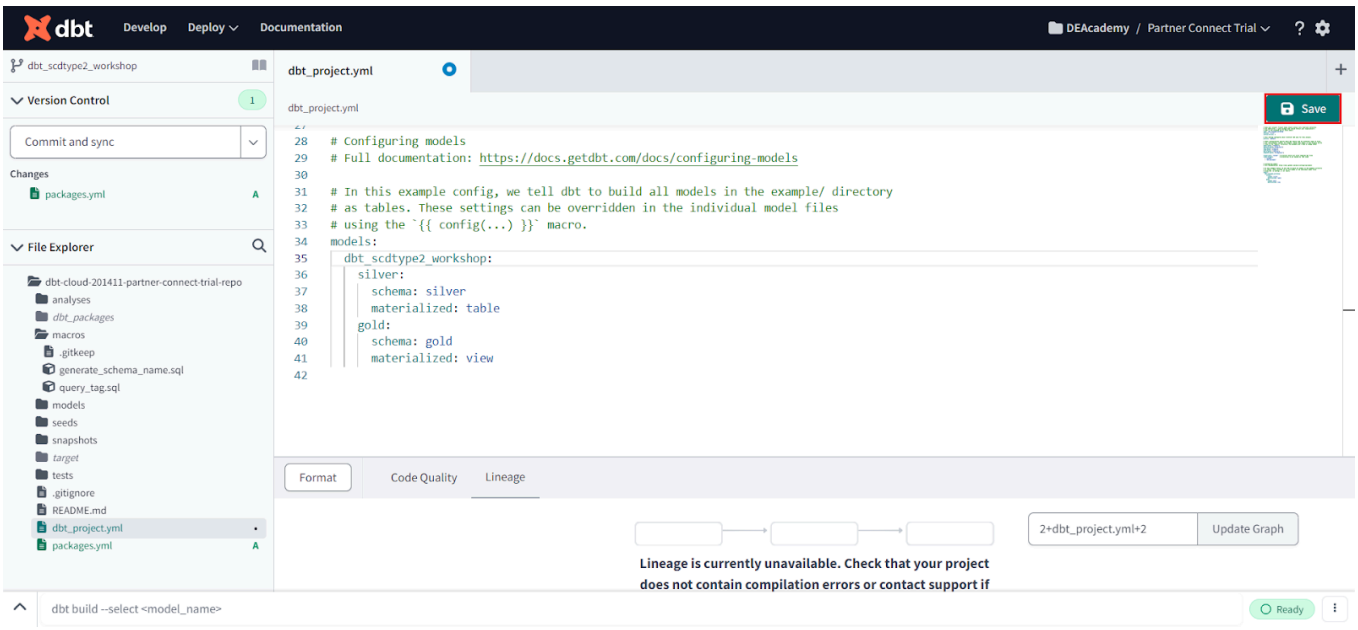
25. After you save the above file, you may get an error. Click on Run “dbt deps” to proceed.



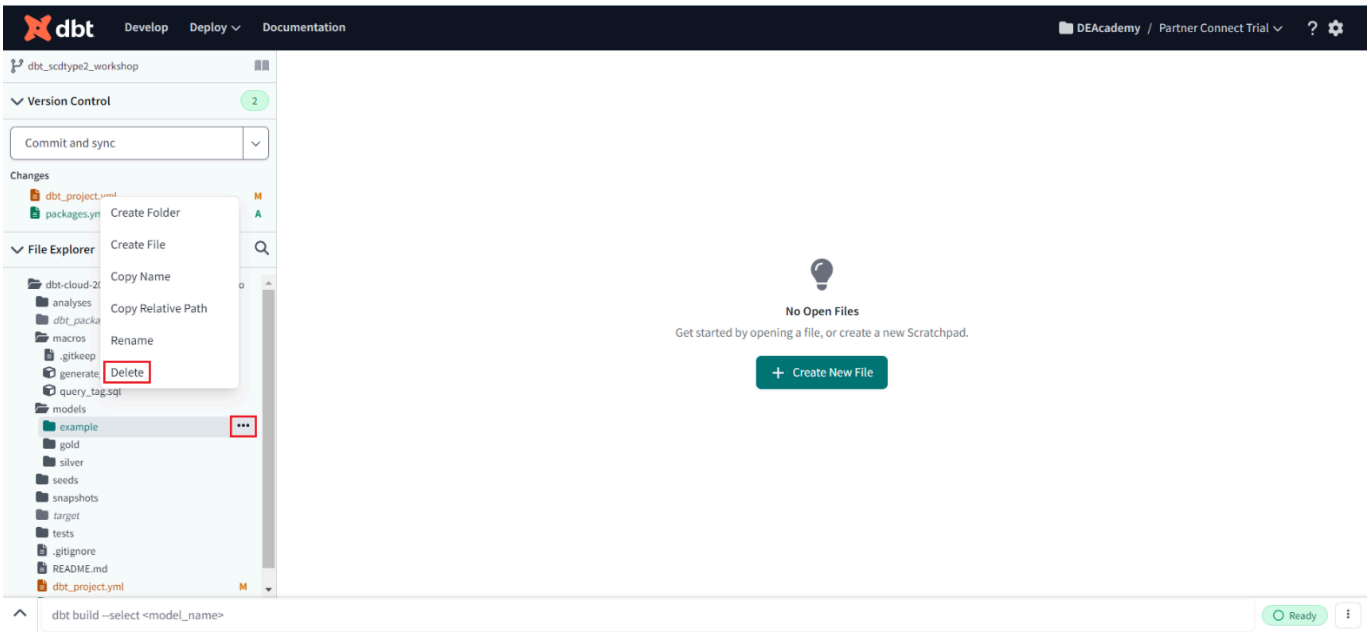
26. Last thing to do before we save our work. Remove the example subdirectory configuration (line 36-37) in the dbt_project.yml file.

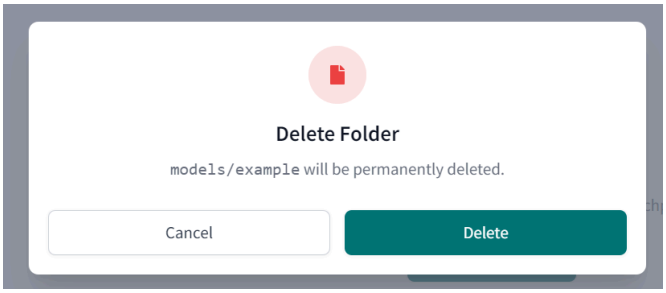


27. Click on save.

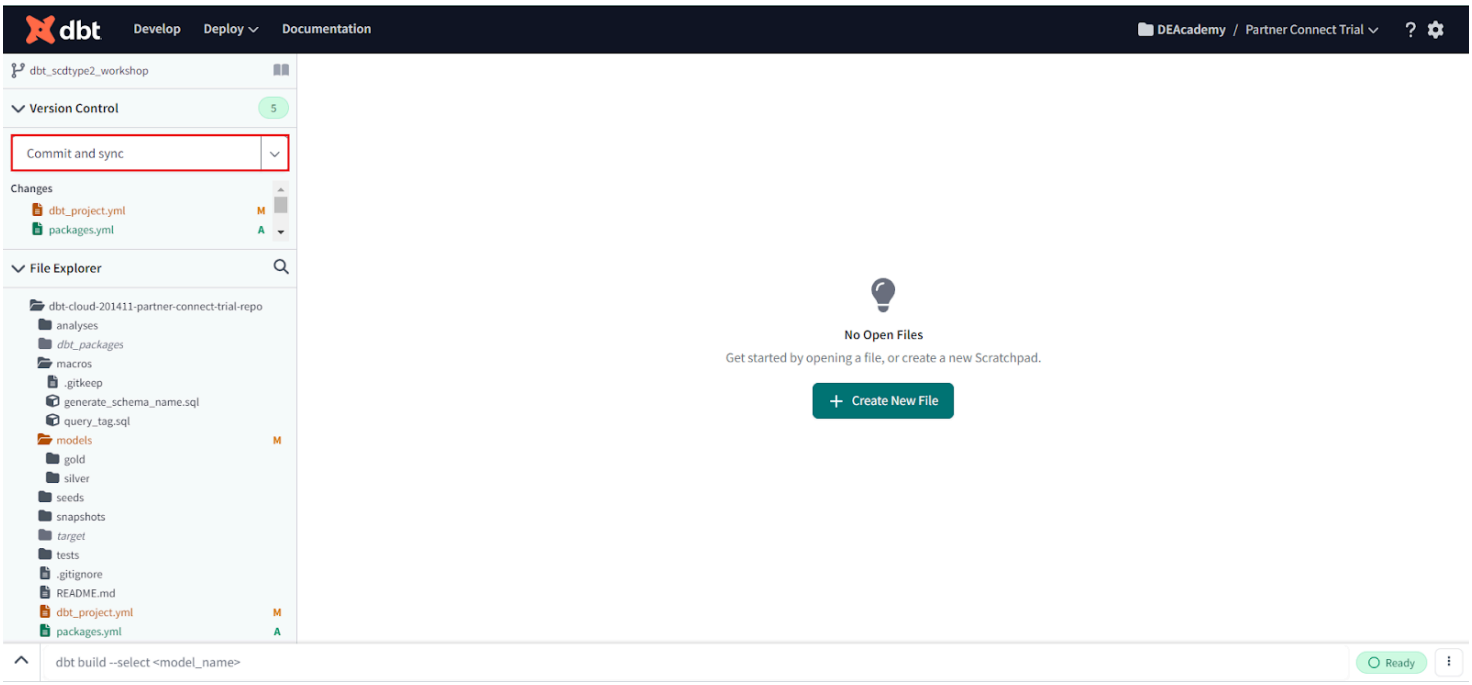


28. Delete the example folder with the models in it. Click on options hovering on example under models. Select Delete and click on Delete.

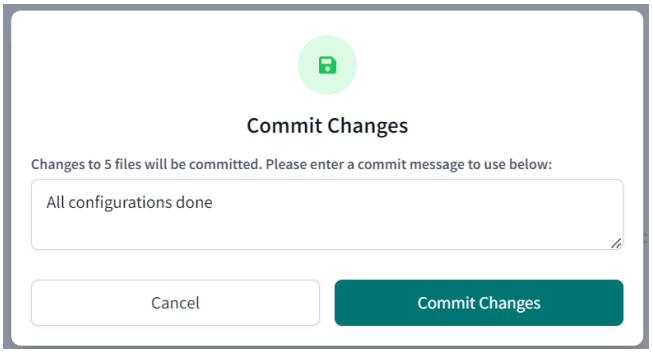




29. Click Commit and sync to commit your work!

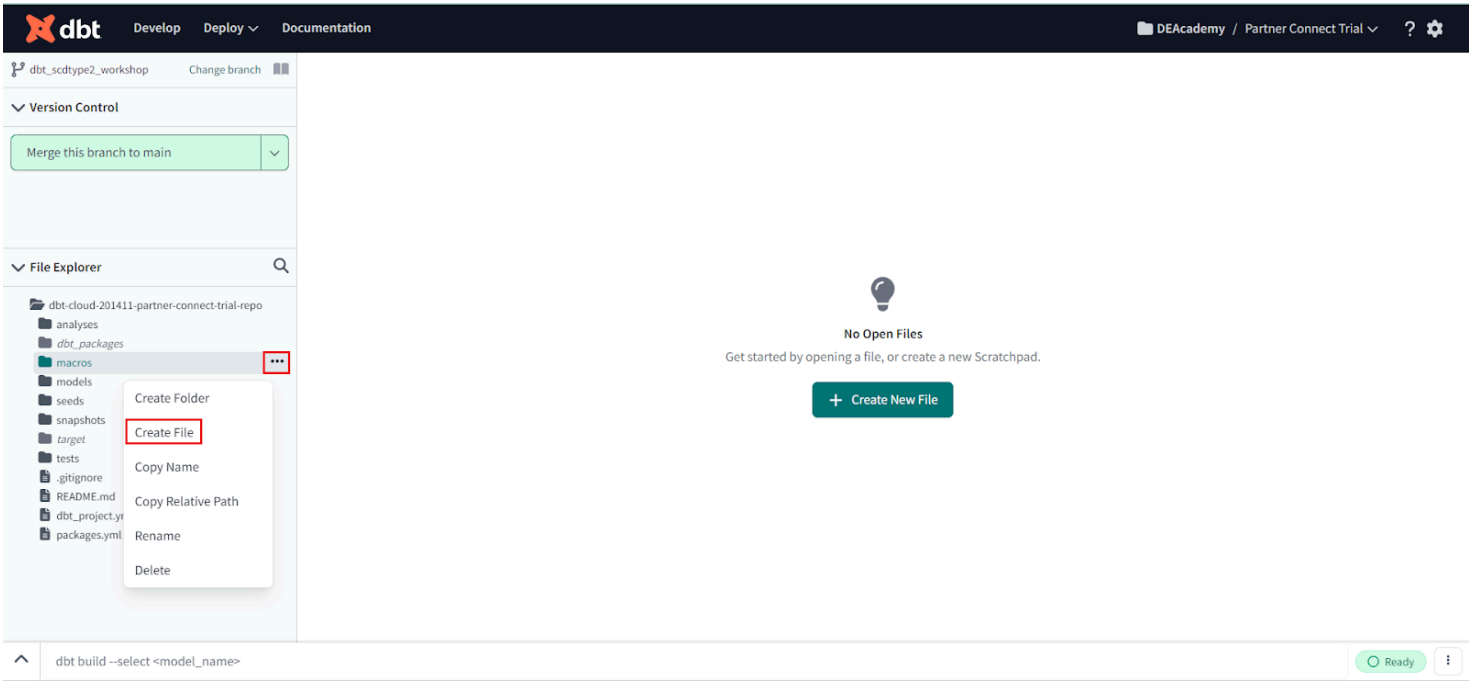


Add a message and click on Commit changes.

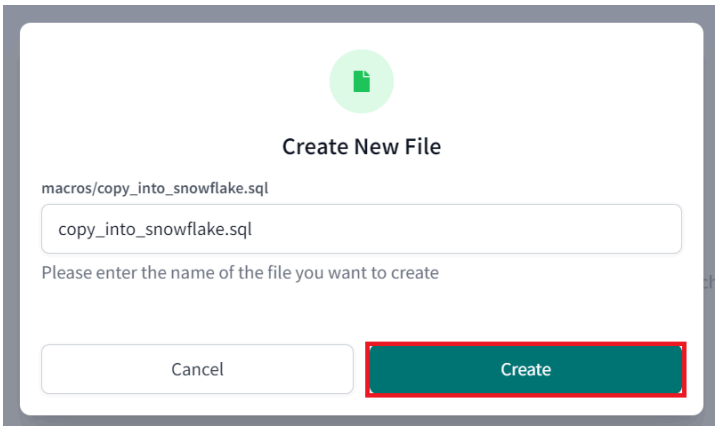


Create Source Integration and Transform

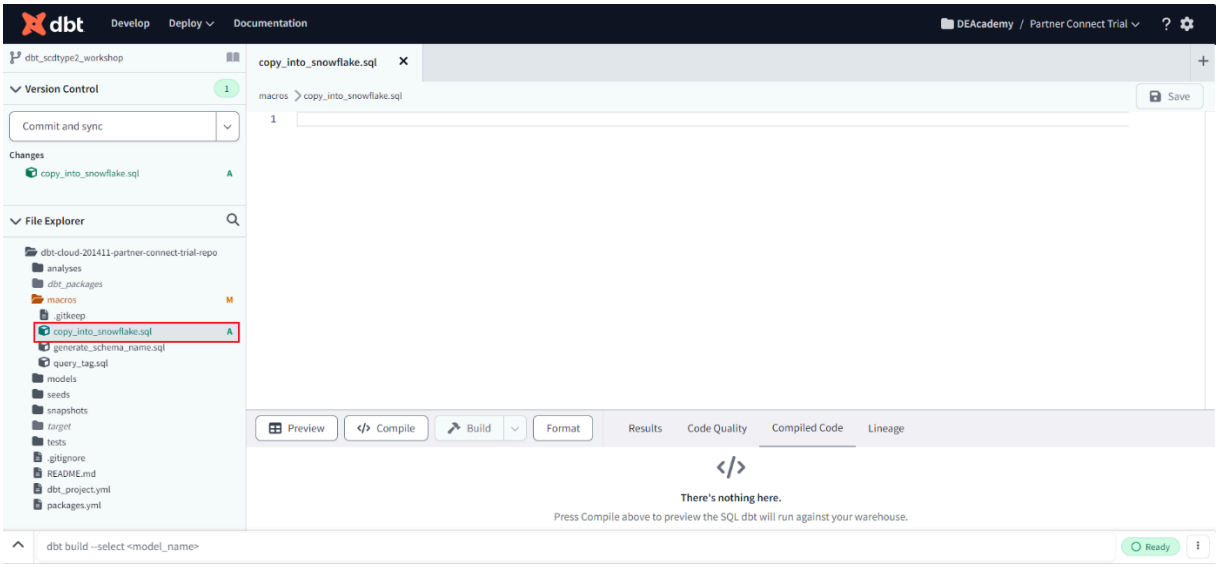
1. We will build a macro to load data from the stage area using COPY INTO command. On the DBT cloud UI, hover on macros and click on '…' and then select Create file.



Name the file as copy_into_snowflake.sql and click on Create.



2. Click on the file and paste the below code:




```
{% macro macros_copy_csv(table_nm) %}

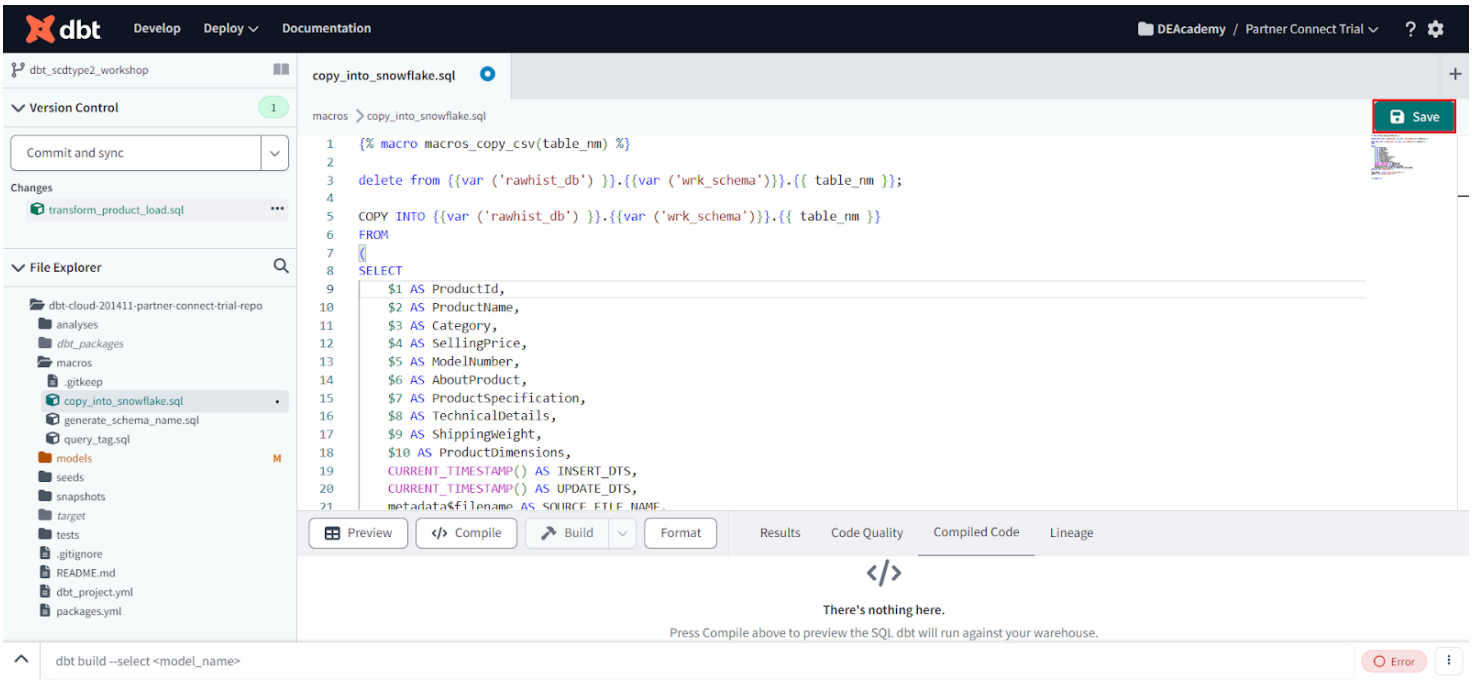
delete from {{var ('rawhist_db') }}.{{var ('wrk_schema')}}.{{ table_nm }};

COPY INTO {{var ('rawhist_db') }}.{{var ('wrk_schema')}}.{{ table_nm }}
FROM
(
SELECT
    $1 AS ProductId,
    $2 AS ProductName,
    $3 AS Category,
    $4 AS SellingPrice,
    $5 AS ModelNumber,
    $6 AS AboutProduct,
    $7 AS ProductSpecification,
    $8 AS TechnicalDetails,
    $9 AS ShippingWeight,
    $10 AS ProductDimensions,
    CURRENT_TIMESTAMP() AS INSERT_DTS,
    CURRENT_TIMESTAMP() AS UPDATE_DTS,
```

```
    metadata$filename AS SOURCE_FILE_NAME,
    metadata$file_row_number AS SOURCE_FILE_ROW_NUMBER
FROM @{{ var('stage_name') }}
)
FILE_FORMAT = {{var ('file_format_json') }}
PURGE={{ var('purge_status') }}
FORCE = TRUE
;


{% endmacro %}
```

Click on Save to proceed.



Note: We will add the vars values in the dbt_project.yml file as assigning in default profile in next step.

3. Click on dbt_project.yml from the left side file pane. Then add the portion of code as shown.

vars:

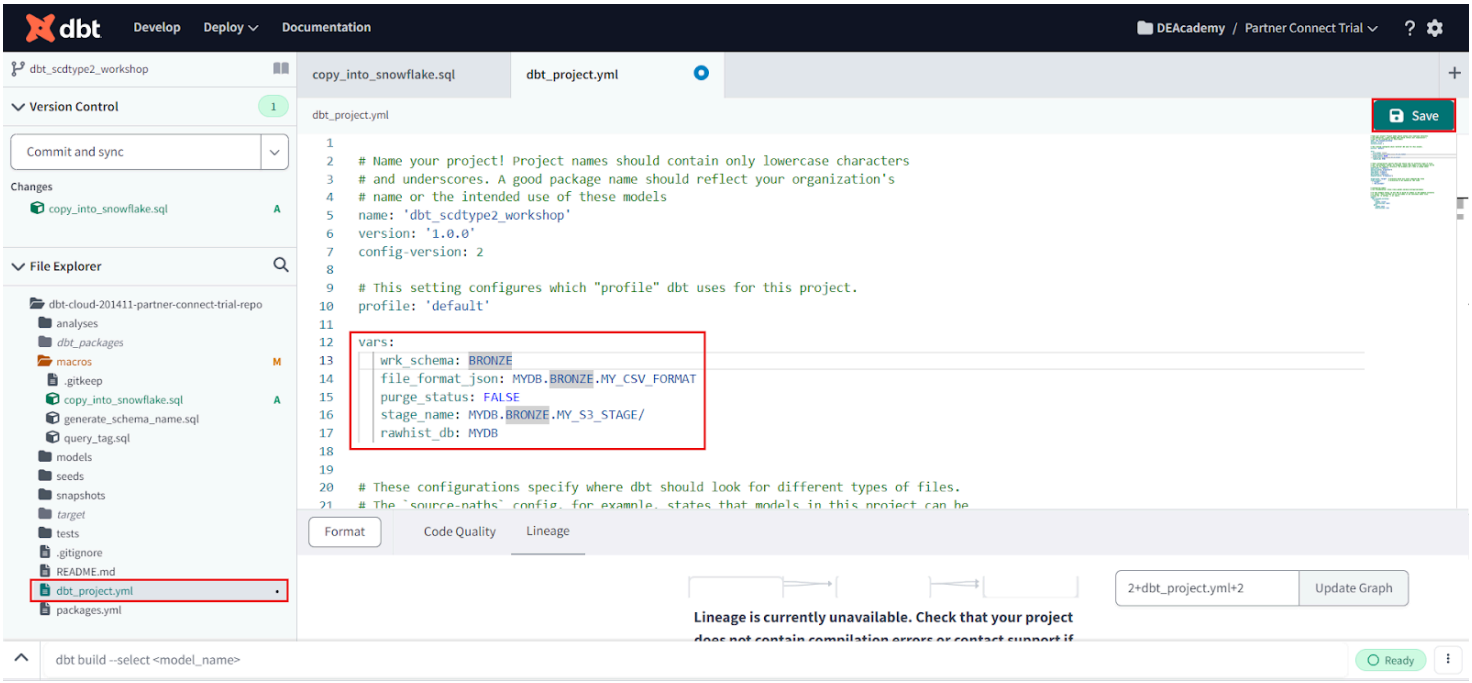
wrk_schema: BRONZE

file_format_json: MYDB.BRONZE.MY_CSV_FORMAT

purge_status: FALSE

stage_name: MYDB.BRONZE.MY_S3_STAGE

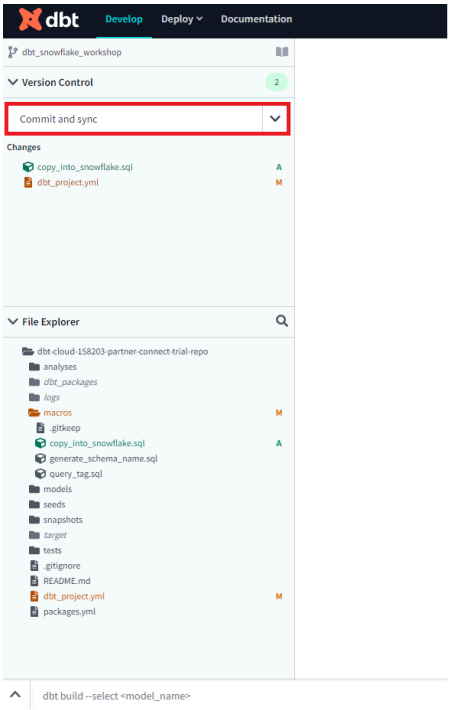
rawhist_db: MYDB

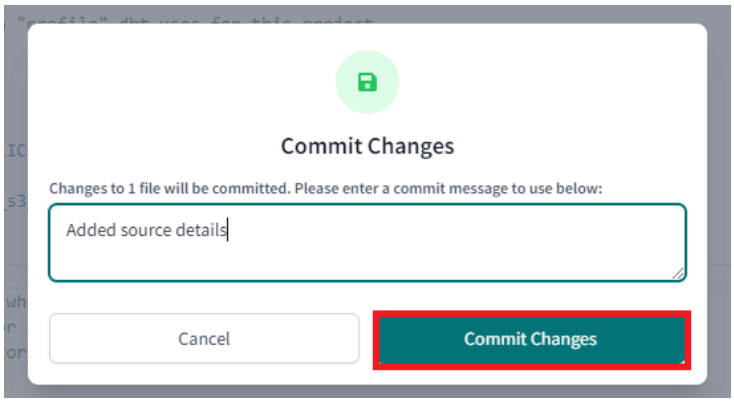


Click on Save to proceed.

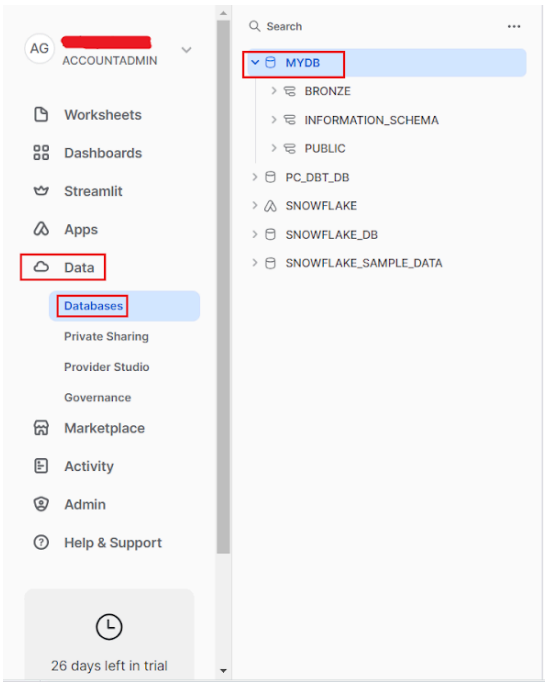
Note: Carefully notice and verify your stage name, file format, work schema and other values, change as per your created or used names in previous process while S3-Snowflake integration.

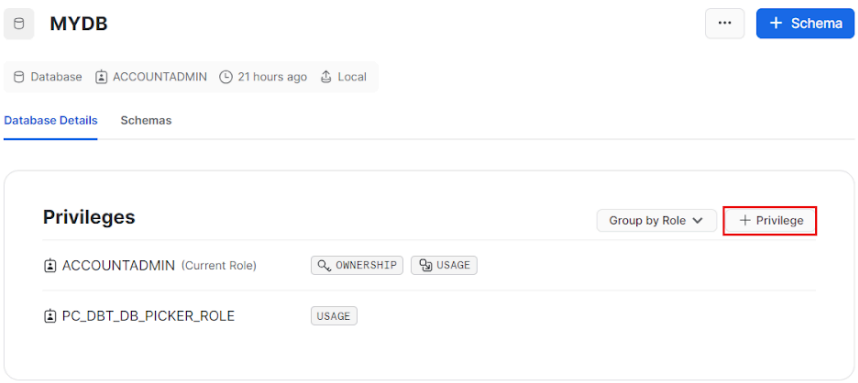
4. Click on Commit and sync to save your work. Add a message and click on Commit changes.

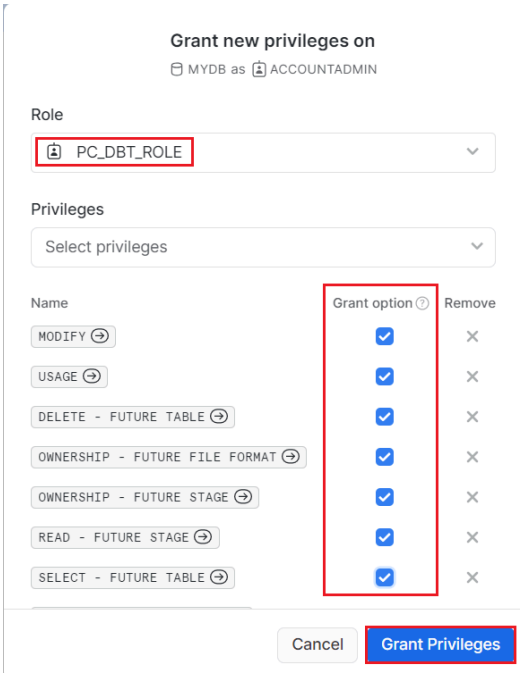




5. Now we need to update some permissions to PC_DBT_ROLE to be able to use MYDB objects. Go to snowflake UI. Click on Data -> Databases -> MYDB -> Click on Privilege from right side of the screen. Update privileges as shown below:

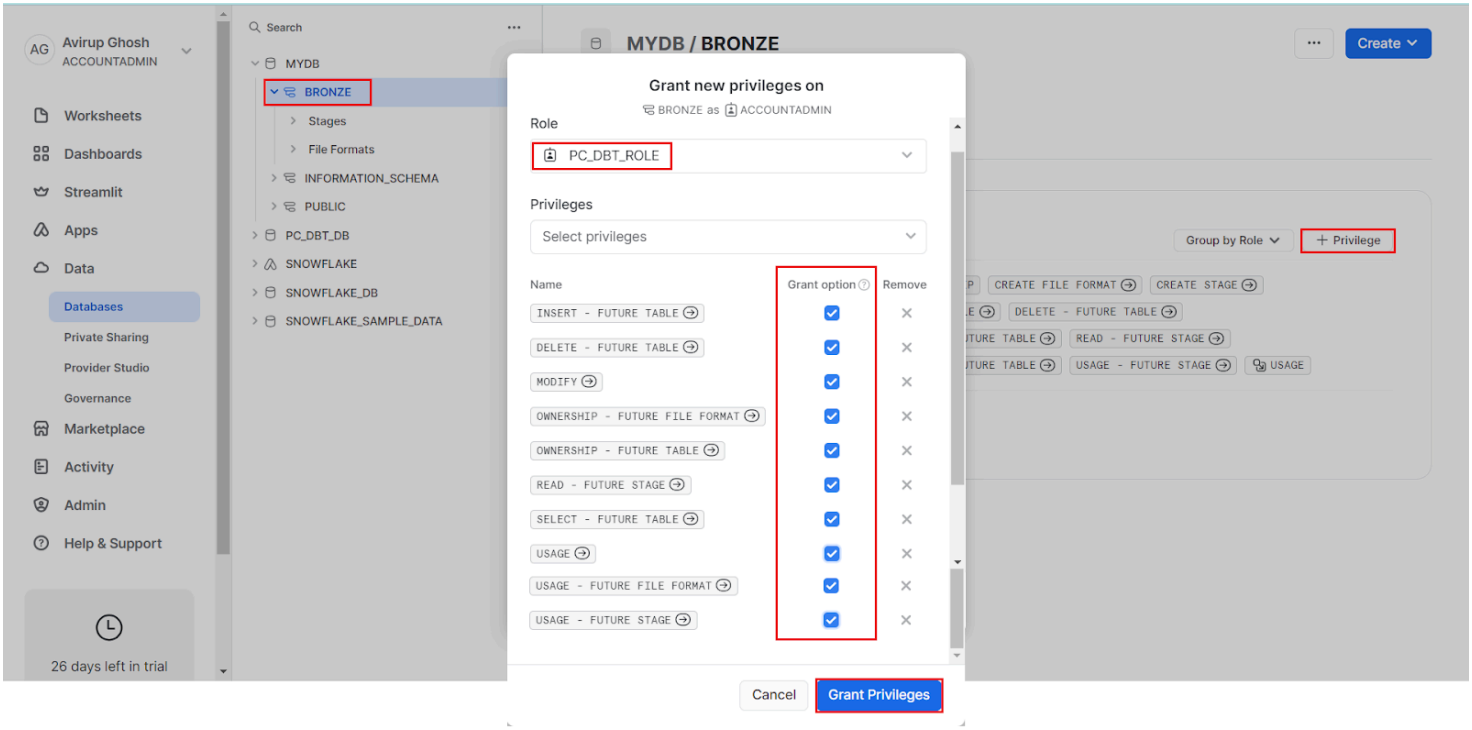






Click on Grant Privileges.

Now do the same for giving permissions for schema BRONZE to PC_DBT_ROLE. Click on Data -> Databases -> MYDB -> BRONZE -> Click on Privilege from right side of the screen like below:



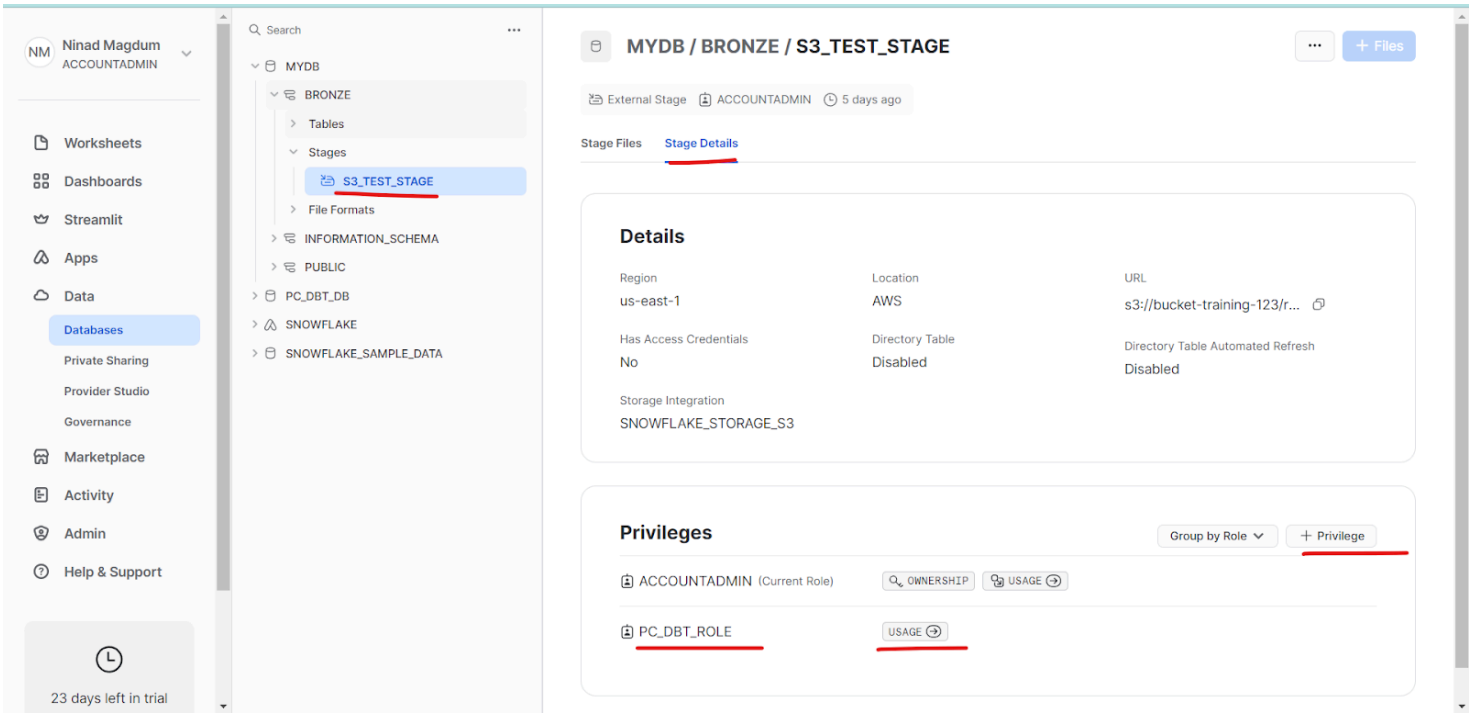
Click on Grant Privileges.

Now do the same for giving permissions for stage S3_TEST_STAGE to PC_DBT_ROLE. Click on Data -> Databases -> MYDB -> BRONZE -> STAGES -> S3_TEST_STAGE -> Click on Stage Details from right side of the screen

Click on Privilege

Select PC_DBT_ROLE

Grant Usage Access



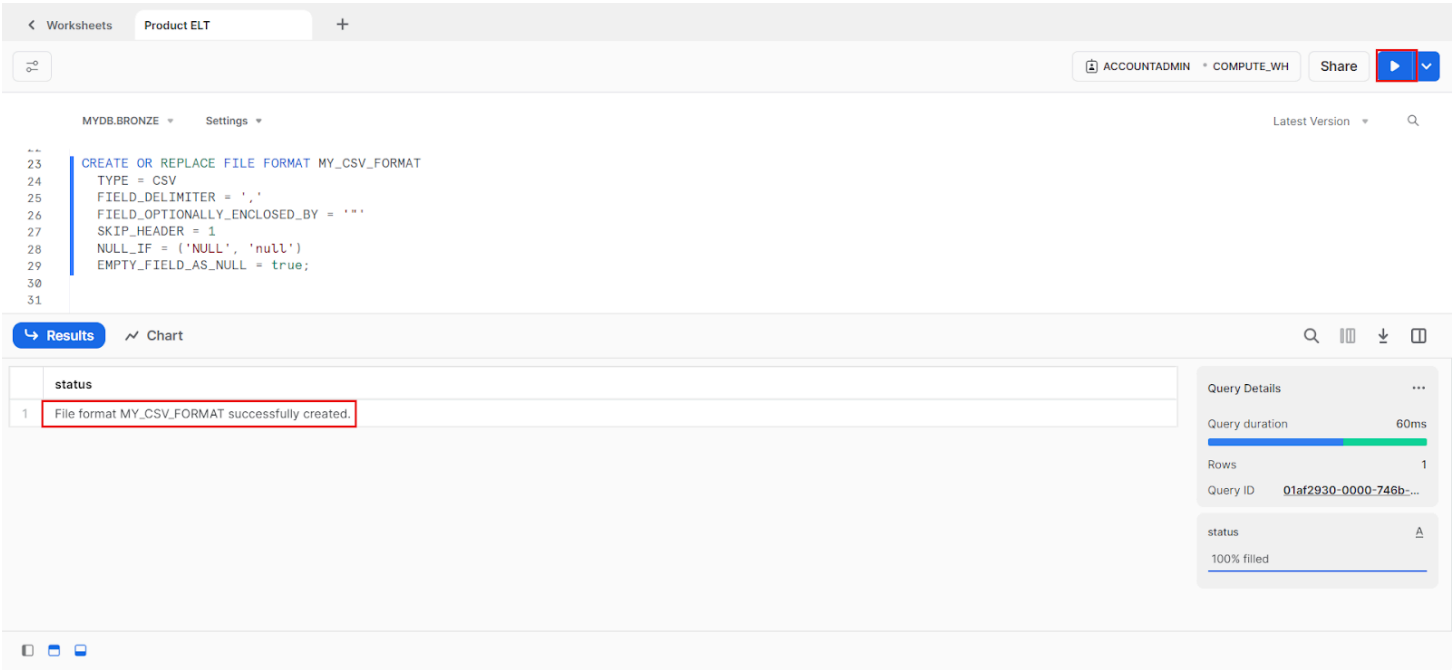
Now go to your worksheets and run the commands below to recreate your stage, file format and new tables as you have just given access to PC_DBT_ROLE to use these. Replace your values as required in <> places.

```
CREATE OR REPLACE FILE FORMAT MY_CSV_FORMAT
  TYPE = CSV
  FIELD_DELIMITER = ','
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
```

```
SKIP_HEADER = 1

NULL_IF = ('NULL', 'null')

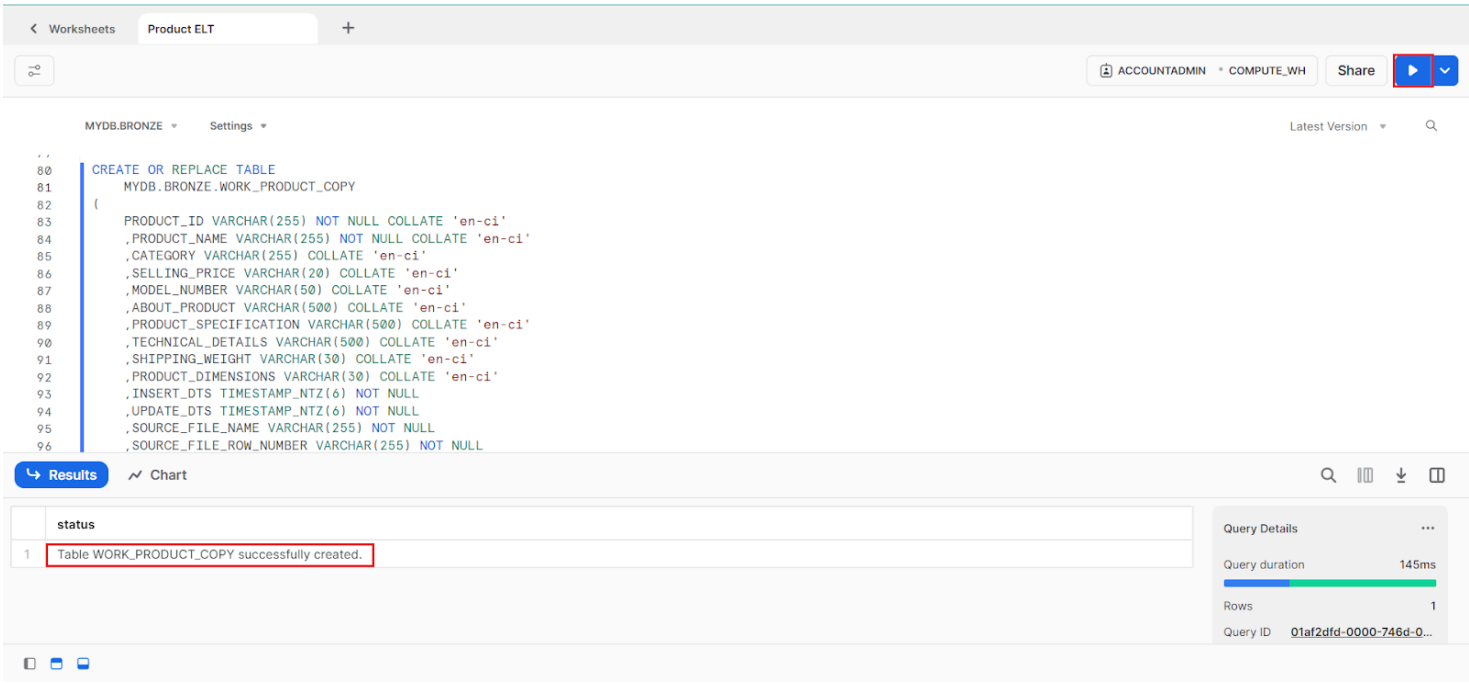
EMPTY_FIELD_AS_NULL = true;
```




```
CREATE OR REPLACE TABLE
    MYDB.BRONZE.WORK_PRODUCT_COPY
(
    PRODUCT_ID VARCHAR(255) NOT NULL COLLATE 'en-ci'
,PRODUCT_NAME VARCHAR(255) NOT NULL COLLATE 'en-ci'
,CATEGORY VARCHAR(255) COLLATE 'en-ci'
,SELLING_PRICE VARCHAR(50) COLLATE 'en-ci'
    ,MODEL_NUMBER VARCHAR(50) COLLATE 'en-ci'
,ABOUT_PRODUCT VARCHAR(5000) COLLATE 'en-ci'
,PRODUCT_SPECIFICATION VARCHAR(5000) COLLATE 'en-ci'
,TECHNICAL_DETAILS VARCHAR(50000) COLLATE 'en-ci'
,SHIPPING_WEIGHT VARCHAR(30) COLLATE 'en-ci'
,PRODUCT_DIMENSIONS VARCHAR(100) COLLATE 'en-ci'
    ,INSERT_DTS TIMESTAMP_NTZ(6) NOT NULL
    ,UPDATE_DTS TIMESTAMP_NTZ(6) NOT NULL
```

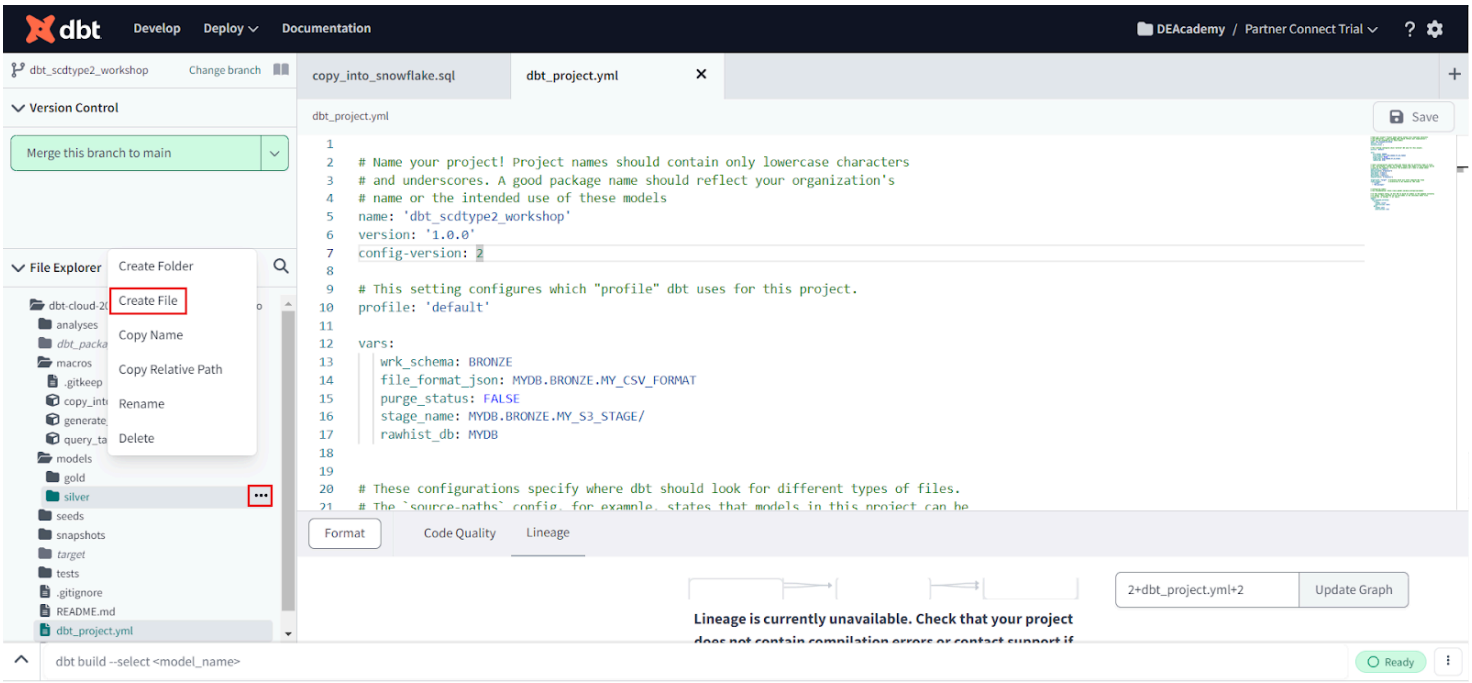
```
,SOURCE_FILE_NAME VARCHAR(255) NOT NULL

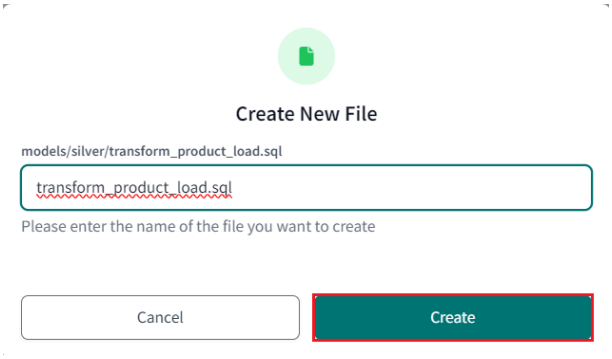
,SOURCE_FILE_ROW_NUMBER VARCHAR(255) NOT NULL);
```



6. Now we will build our target models to transform the copied data in _COPY. Follow and paste the codes below to create one transform table.

Let us create a transform model under models/silver named transform_product_load.sql. Click on Create to proceed.





Now open the created file by clicking on models -> silver -> transform_product_load.sql. Paste the below code to load data from the staged files into a transient table.

```
{{ config({ "materialized":'table',

 "transient":true,

 "alias":'WORK_PRODUCT_TRANSFORM',

 "pre_hook": macros_copy_csv('WORK_PRODUCT_COPY'),

 "schema": 'SILVER'

})}}


WITH transform AS(

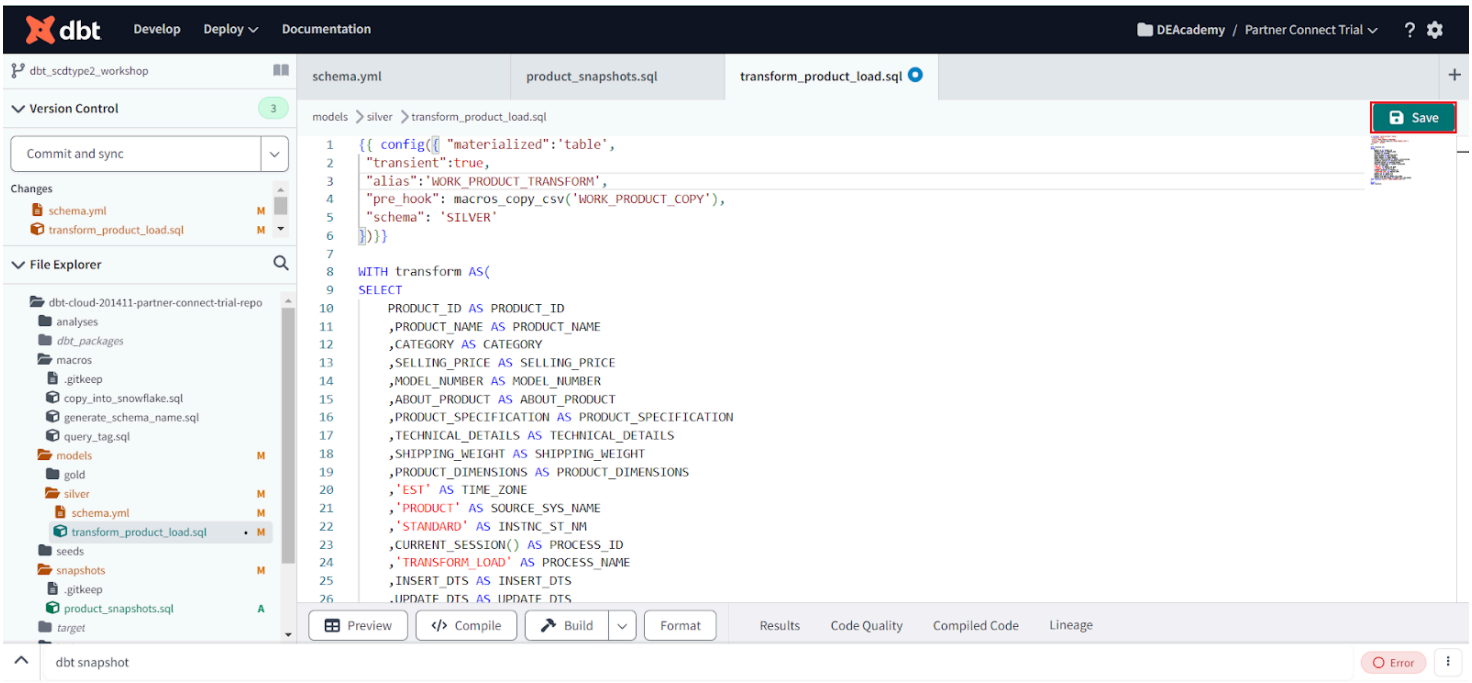
SELECT

    PRODUCT_ID AS PRODUCT_ID
```

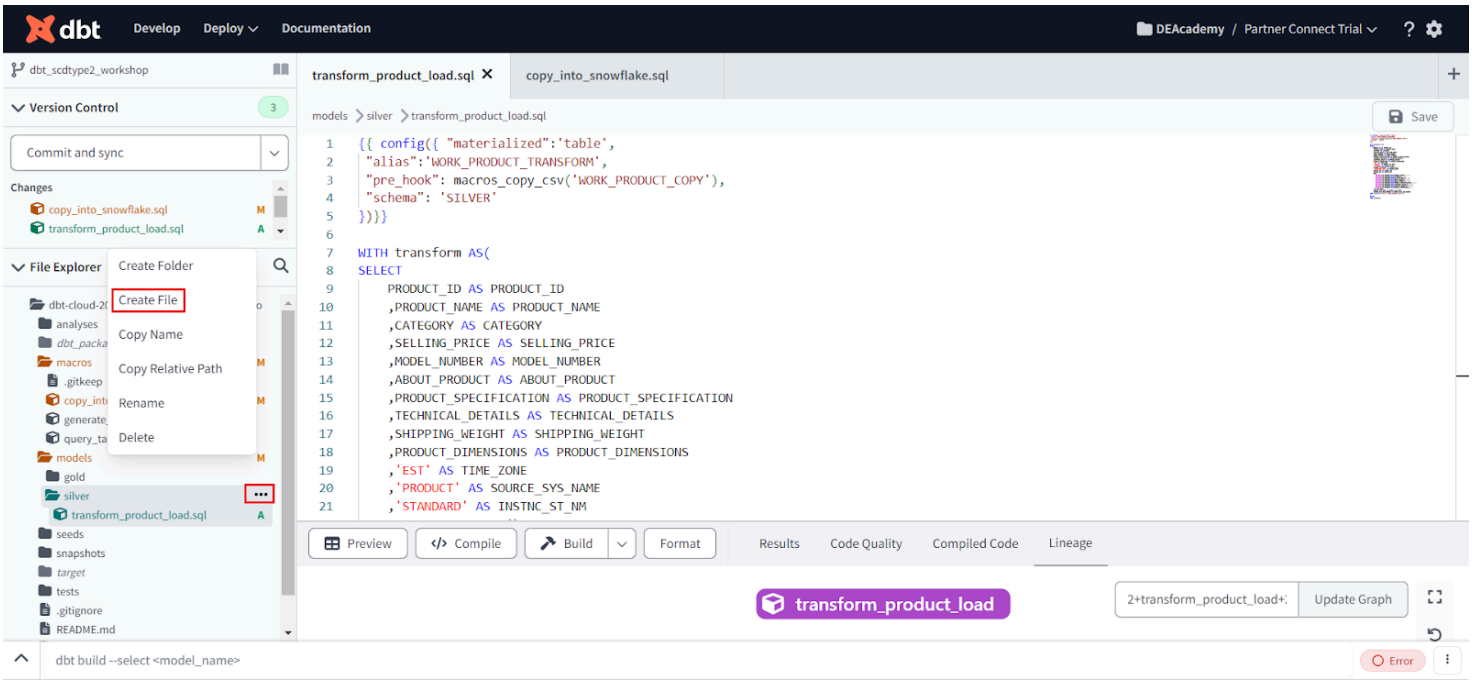
```
    ,PRODUCT_NAME AS PRODUCT_NAME
    ,CATEGORY AS CATEGORY
    ,SELLING_PRICE AS SELLING_PRICE
    ,MODEL_NUMBER AS MODEL_NUMBER
    ,ABOUT_PRODUCT AS ABOUT_PRODUCT
    ,PRODUCT_SPECIFICATION AS PRODUCT_SPECIFICATION
    ,TECHNICAL_DETAILS AS TECHNICAL_DETAILS
    ,SHIPPING_WEIGHT AS SHIPPING_WEIGHT
    ,PRODUCT_DIMENSIONS AS PRODUCT_DIMENSIONS
    ,'EST' AS TIME_ZONE
    ,'PRODUCT' AS SOURCE_SYS_NAME
    ,'STANDARD' AS INSTNC_ST_NM
    ,CURRENT_SESSION() AS PROCESS_ID
    ,'TRANSFORM_LOAD' AS PROCESS_NAME
    ,INSERT_DTS AS INSERT_DTS
    ,UPDATE_DTS AS UPDATE_DTS
    ,SOURCE_FILE_NAME AS SOURCE_FILE_NAME
    ,SOURCE_FILE_ROW_NUMBER AS SOURCE_FILE_ROW_NUMBER
FROM {{source('source','WORK_PRODUCT_COPY')}}
)

SELECT *
FROM transform
```

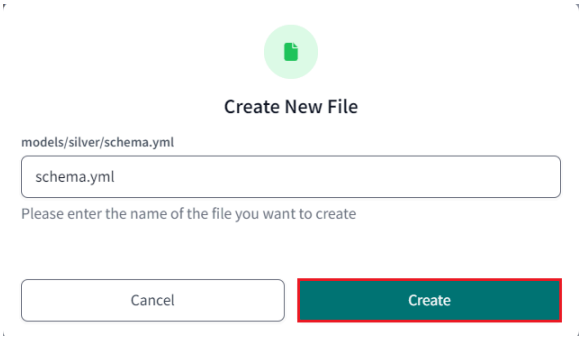
Click on Save to proceed.



7. Now create a schema file under models/silver named schema.yml. Click on Create File to proceed.

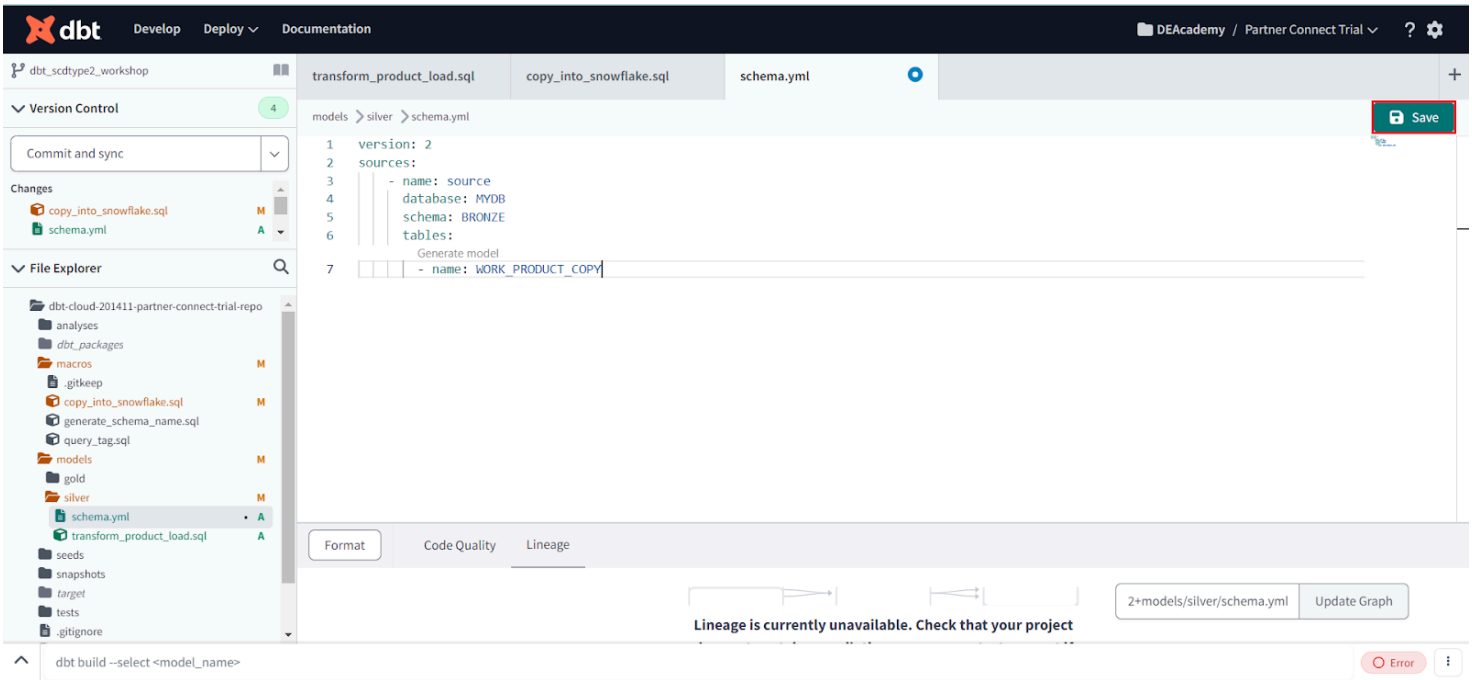


Give the file name and click on Create.

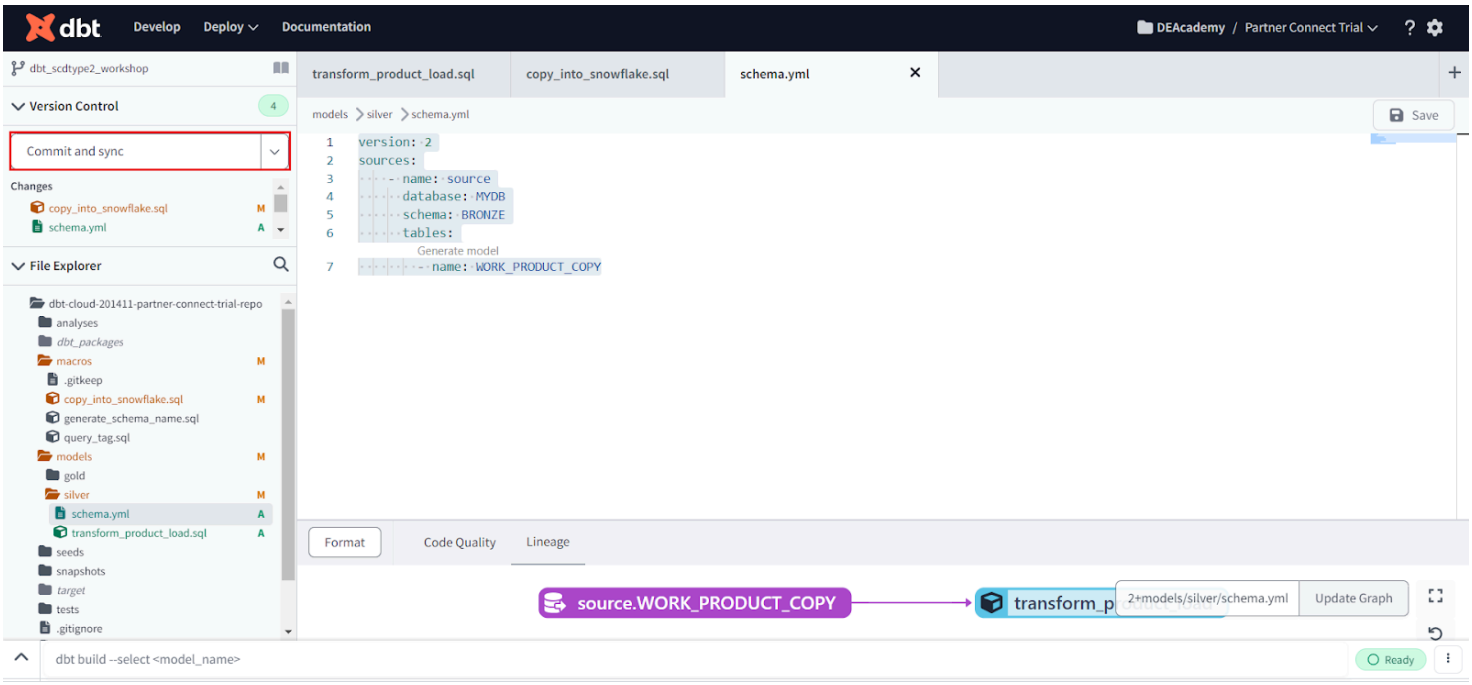


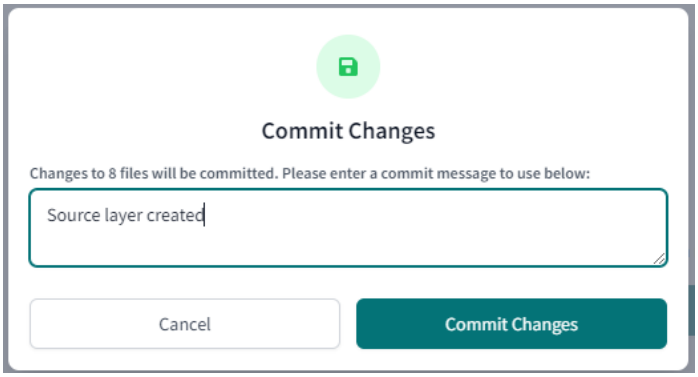
8. Add below details to the file and click on Save to proceed.

```
version: 2
sources:
  - name: source
    database: MYDB
    schema: BRONZE
    tables:
      - name: WORK_PRODUCT_COPY
```

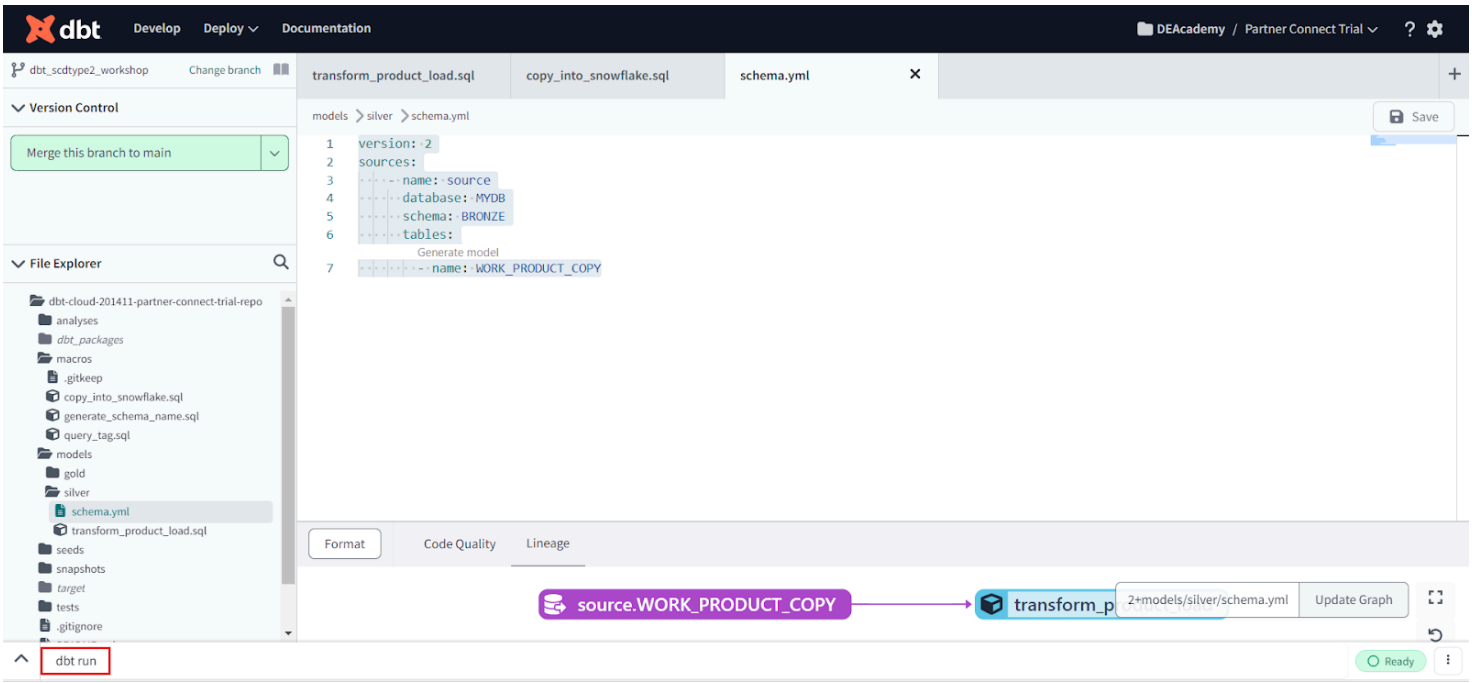


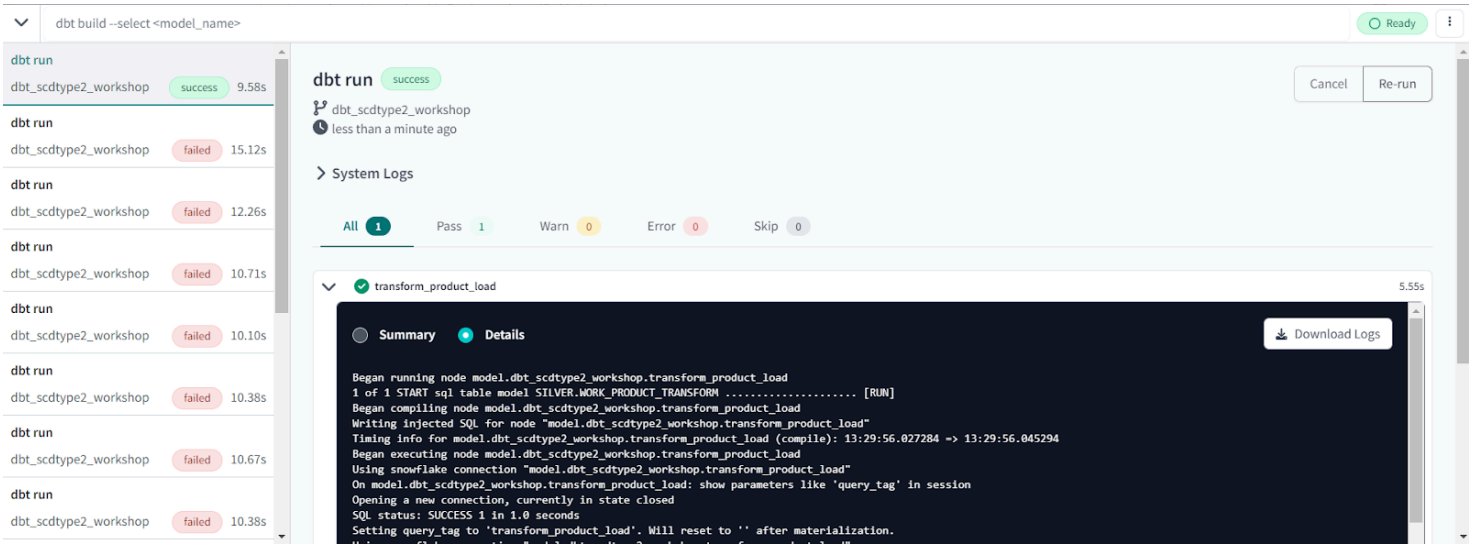
9. Please click on Commit and sync after all sources are completed to keep the work saved and progressing. Add a message and click on Commit changes.



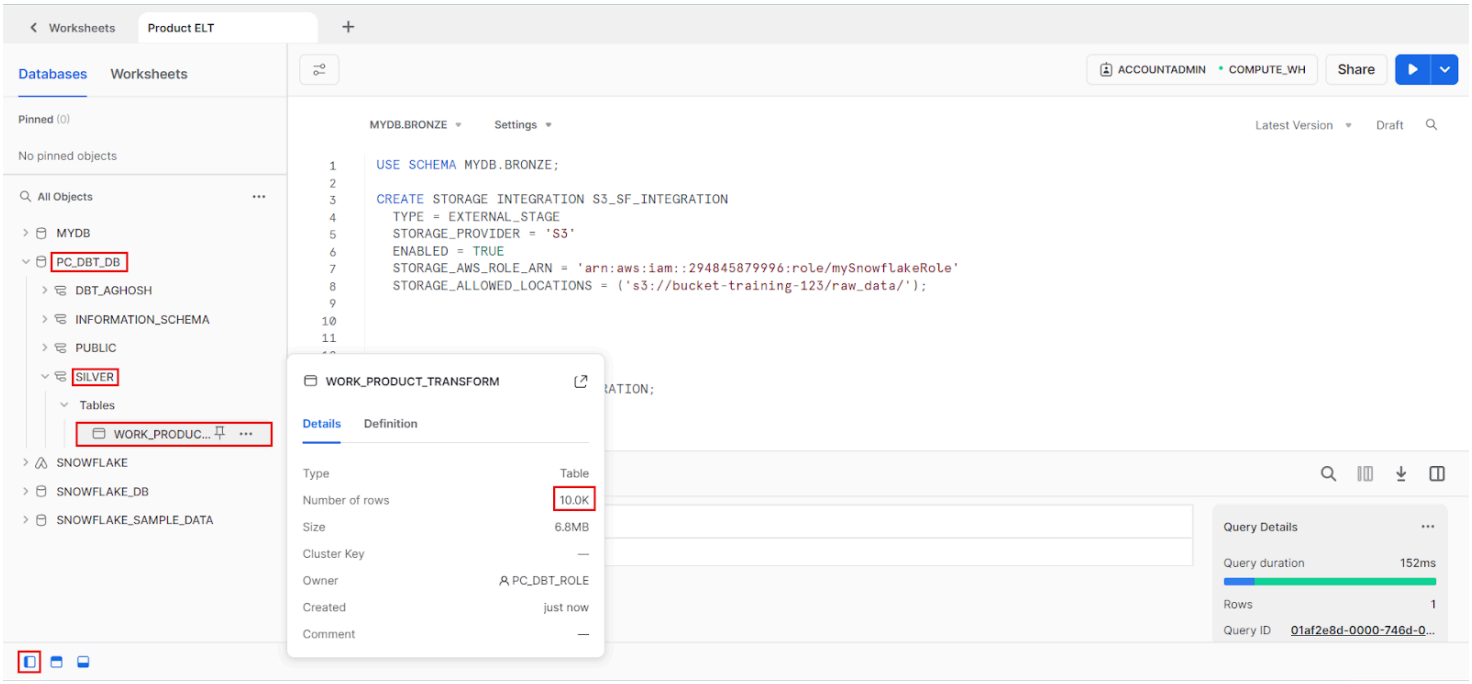


10. After the commit, now type dbt run in the command line to run all your models. Hit Enter on your keyboard to run.





11. Go to your Snowflake worksheet and from the left side object pane, open the Database and Schema as shown to verify the data load.



12. Now open the schema.yml file in DBT cloud from model/silver folder and add a table like below. Then click on Save to proceed.

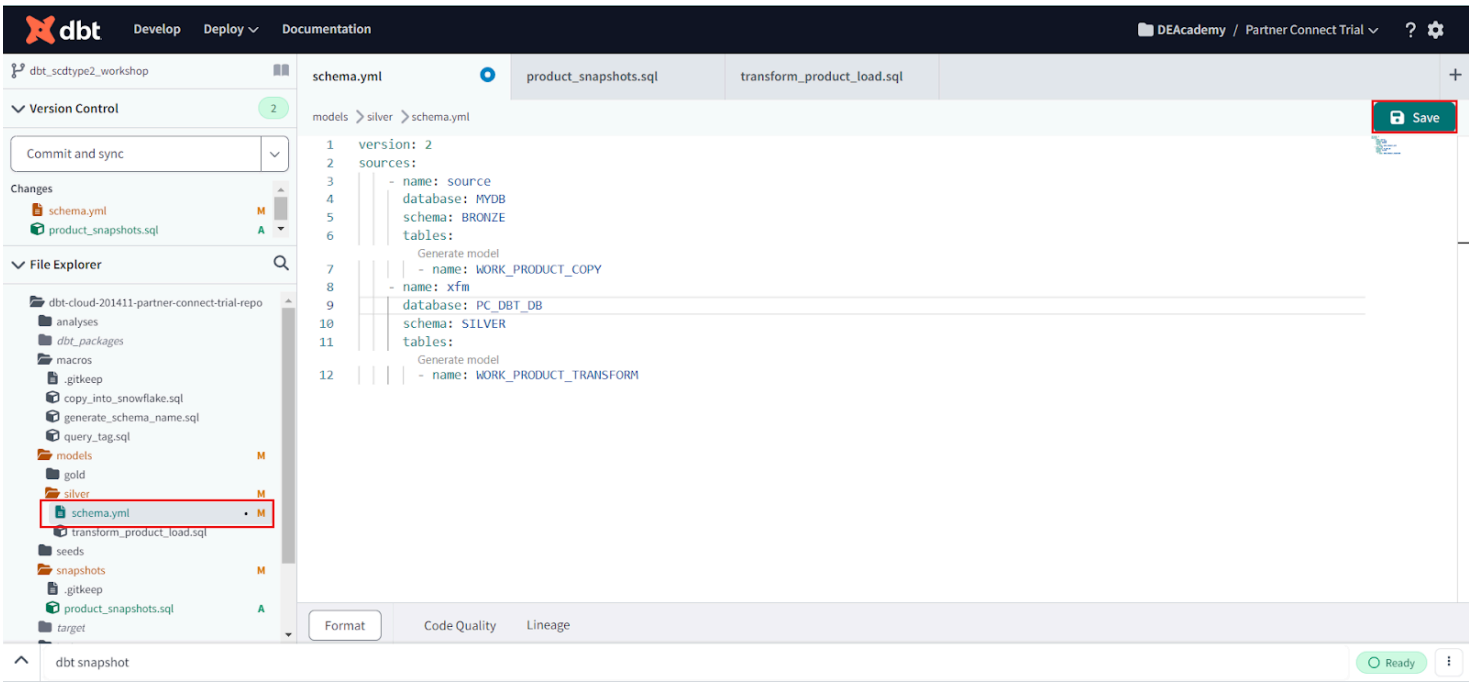
```
- name: xfm

    database: PC_DBT_DB

    schema: SILVER

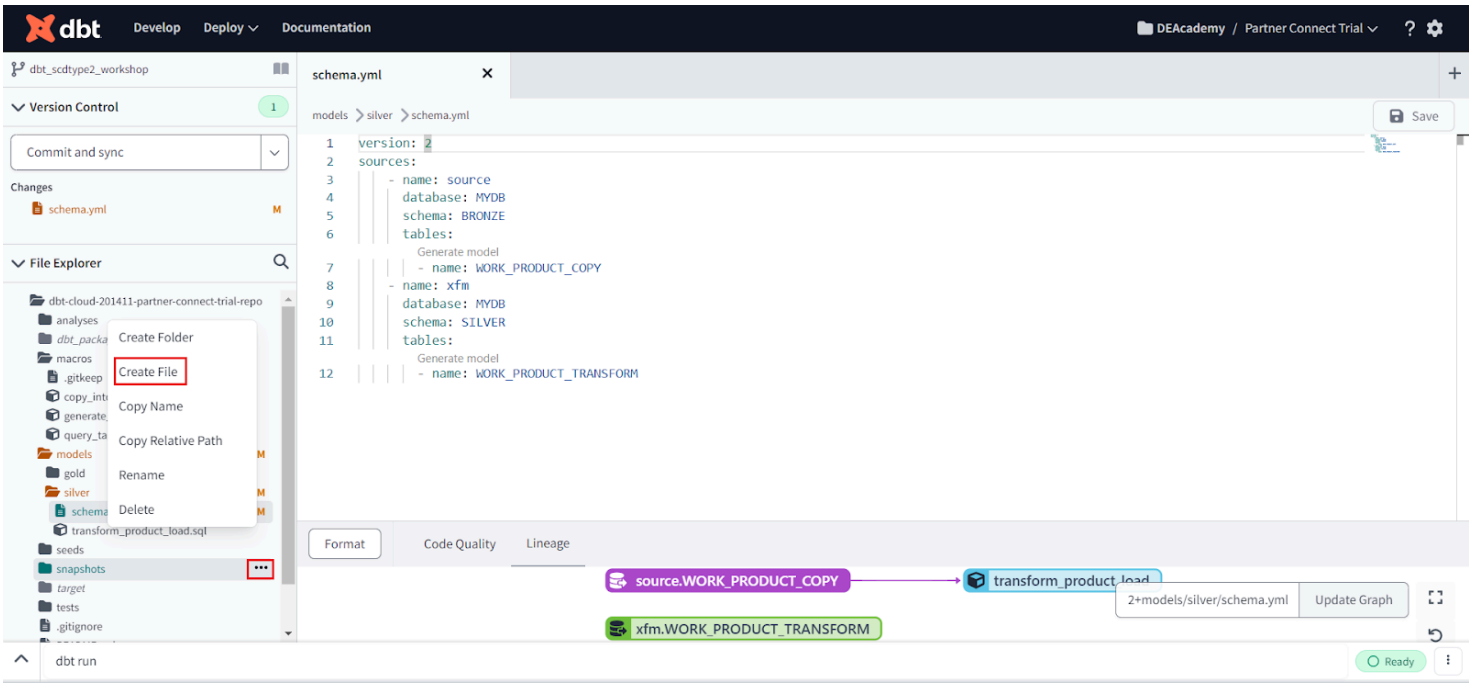
    tables:

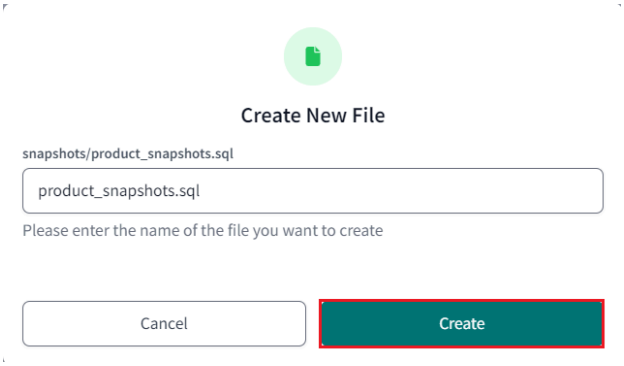
      - name: WORK_PRODUCT_TRANSFORM
```



Create Snapshots

1. Click on the snapshots folder and use the '…' to open options and then click on Create File. Give a name to the file like product_snapshots.sql.





2. Now add the below details to create a snapshot based on an unique_key and all other changeable columns with respect to data. Once done click on Save to proceed.

```
{% snapshot product_snapshot %}

{{

    config(

      target_database='PC_DBT_DB',

      target_schema='snapshots',

      unique_key='PRODUCT_ID',

      strategy='check',

      check_cols=['PRODUCT_NAME', 'CATEGORY', 'SELLING_PRICE', 'MODEL_NUMBER',
'ABOUT_PRODUCT', 'PRODUCT_SPECIFICATION',

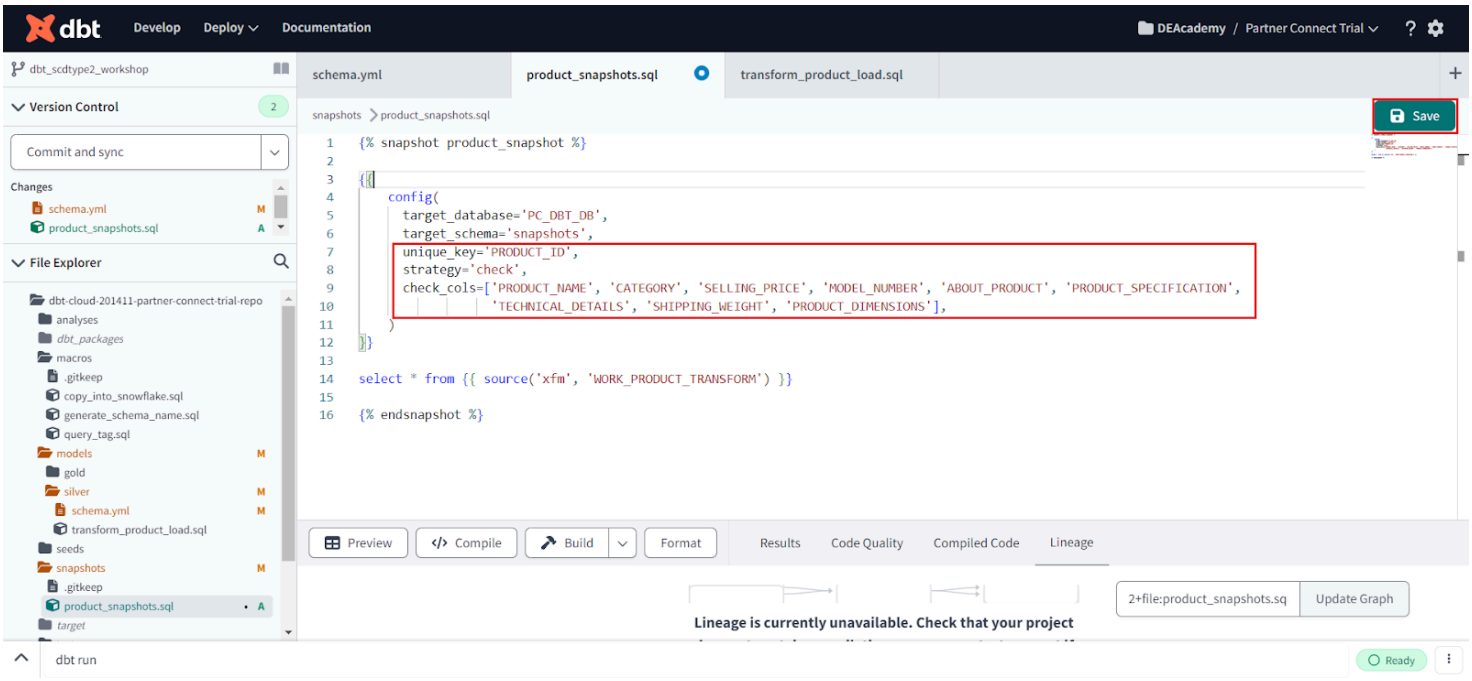
                'TECHNICAL_DETAILS', 'SHIPPING_WEIGHT', 'PRODUCT_DIMENSIONS'],

    )

}}

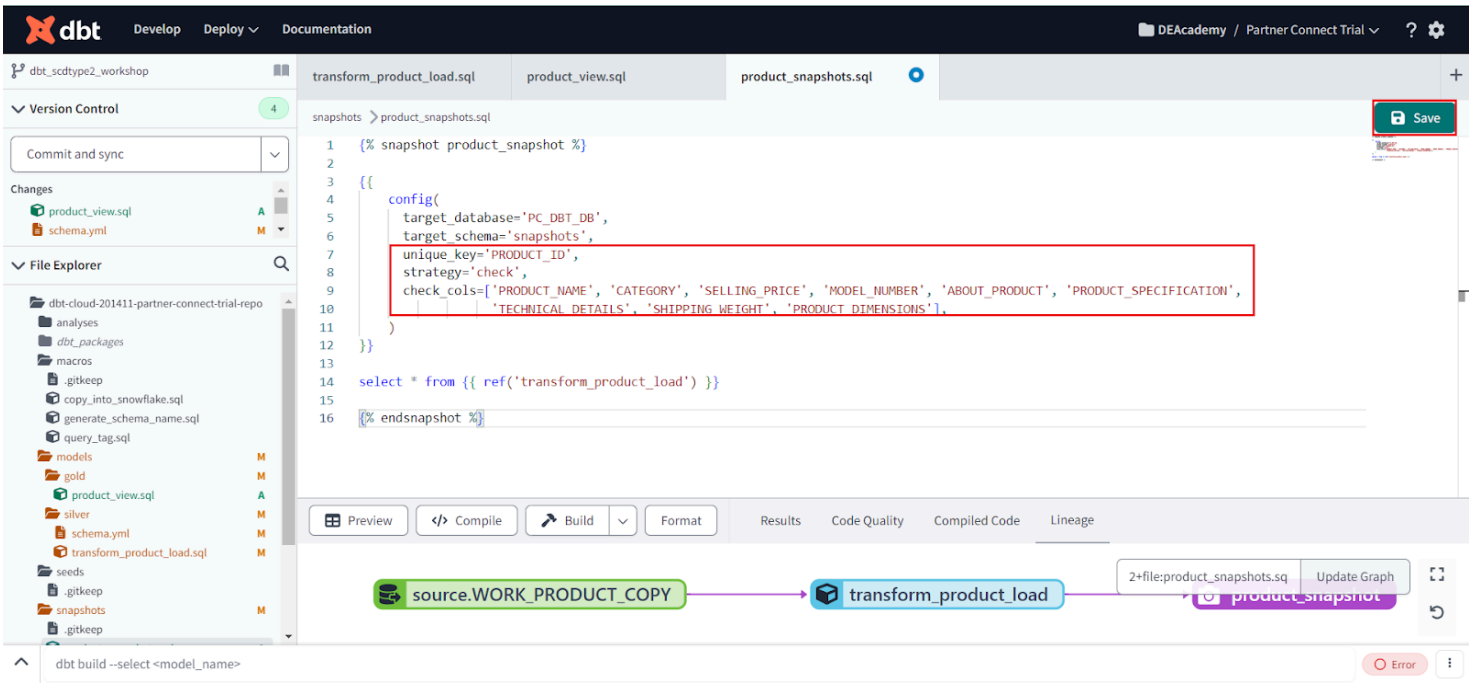
select * from {{ source('xfm', 'WORK_PRODUCT_TRANSFORM') }}

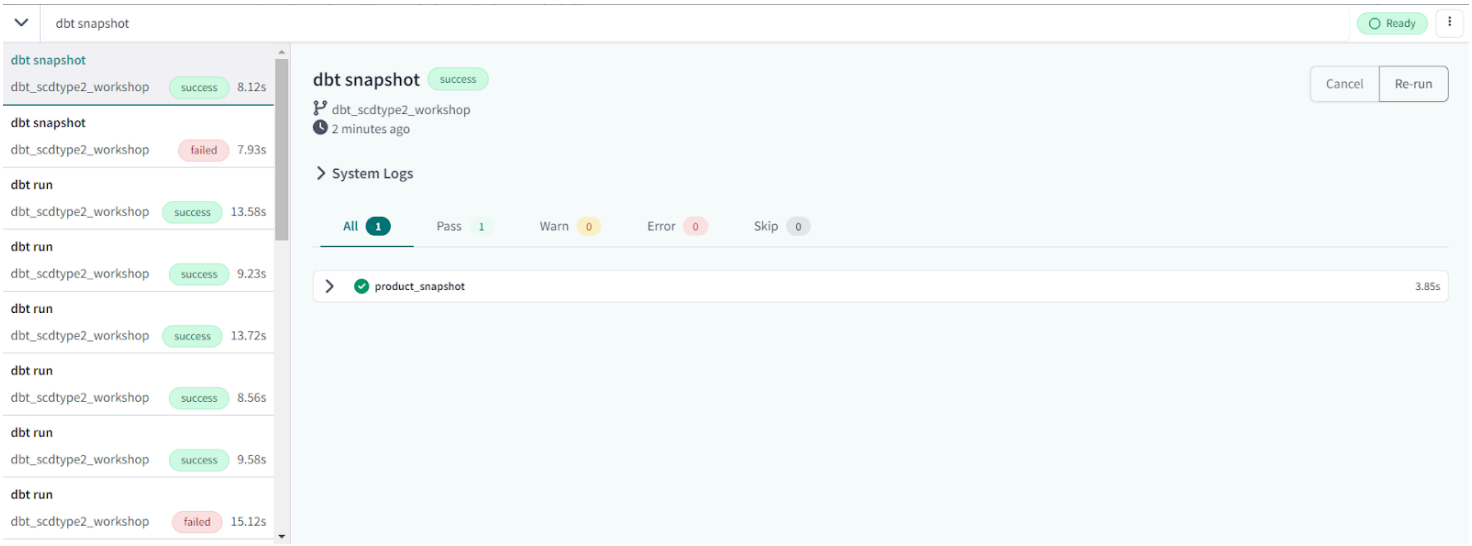
{% endsnapshot %}
```



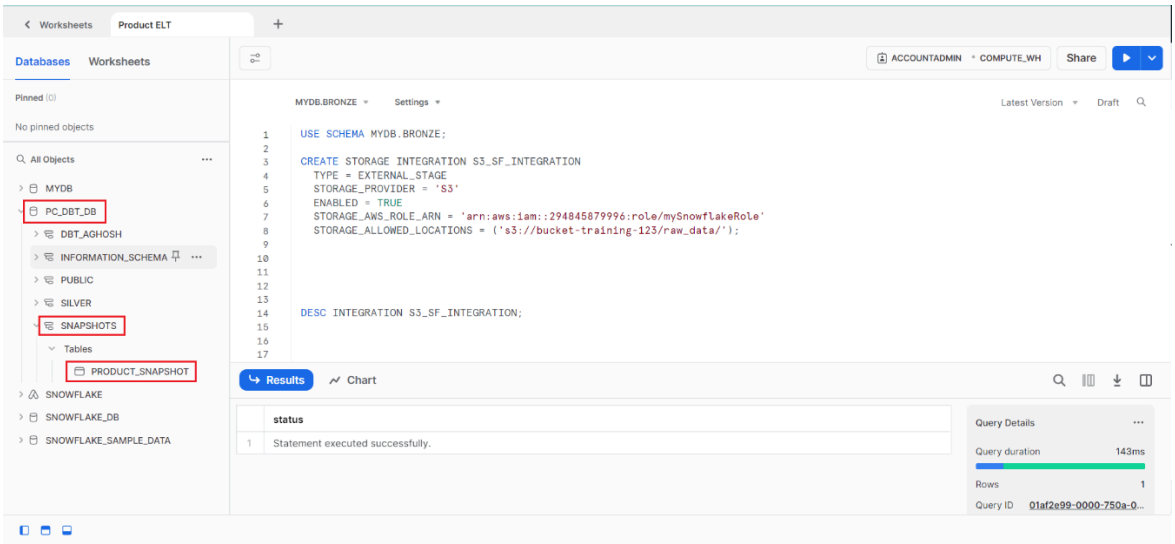
Notice we are using the recently added source from the schema.yml file in above step. Also after saving you can see the lineage.

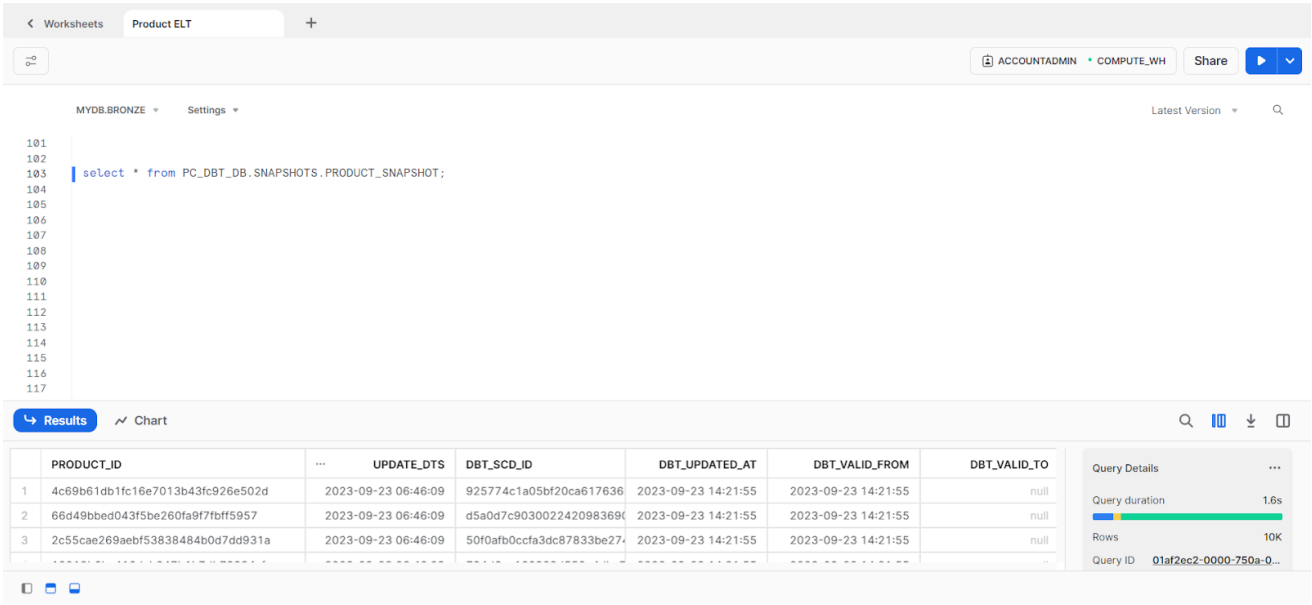
3. Now let us run dbt snapshot command and see the data captured using snapshot. Hit Enter on your keyboard to run.





4. Now go to your snowflake worksheet and from left side objects pane find the schema snapshots. Run a simple select to see the data like below.

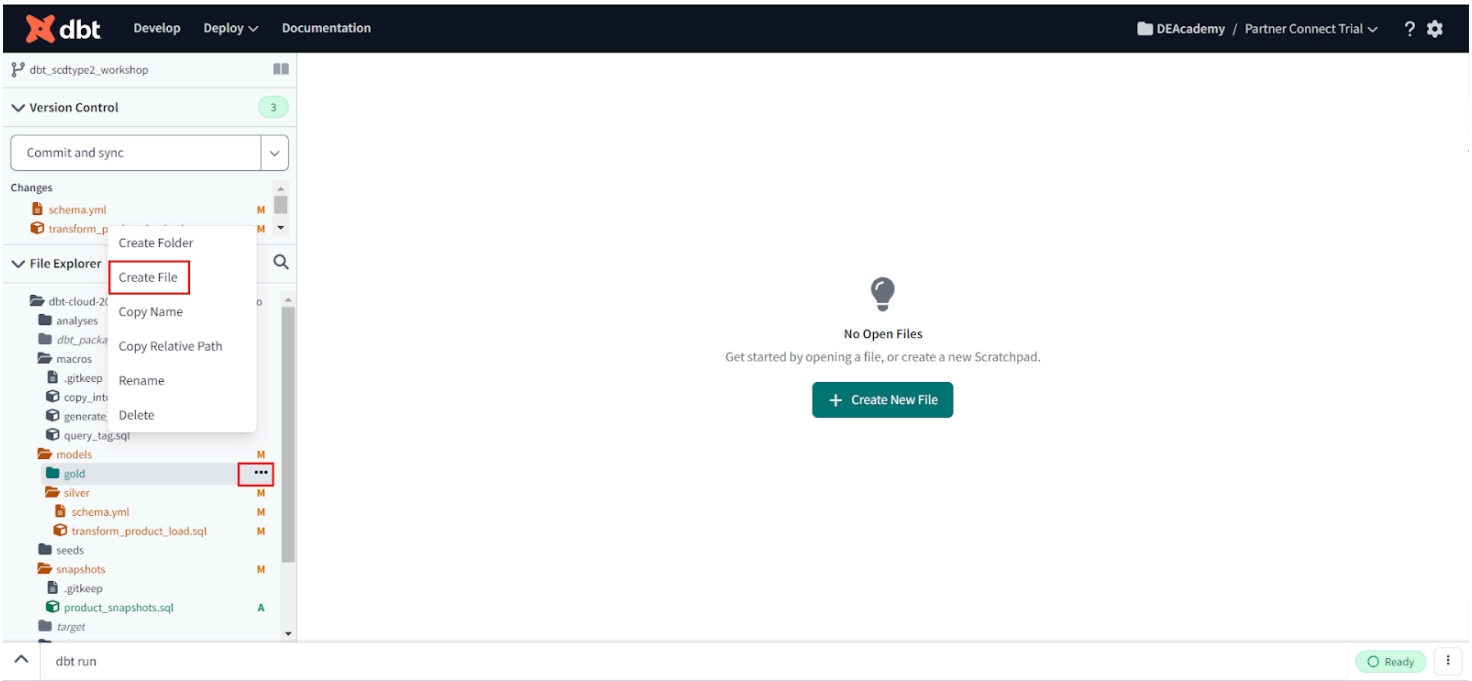


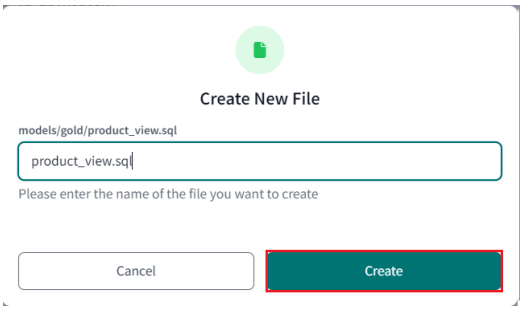


Notice the DBT created columns to capture the change in data using check strategy for each unique key Product_ID.

Create View in Gold Schema

1. Now hover on gold folder and the click on … and then click on Create File. Give it a name like product_view.sql. Then click on Create.



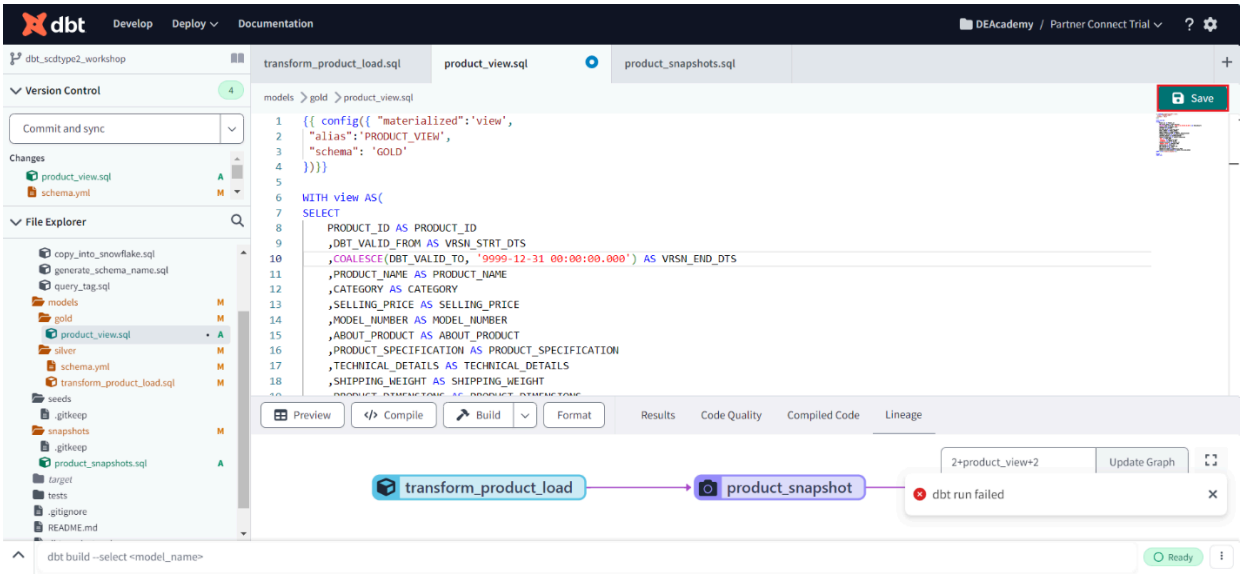


2. Now add the below script for the model and then click on Save.

```
{{ config({ "materialized":'view',
 "alias":'PRODUCT_VIEW',
 "schema": 'GOLD'
})}}

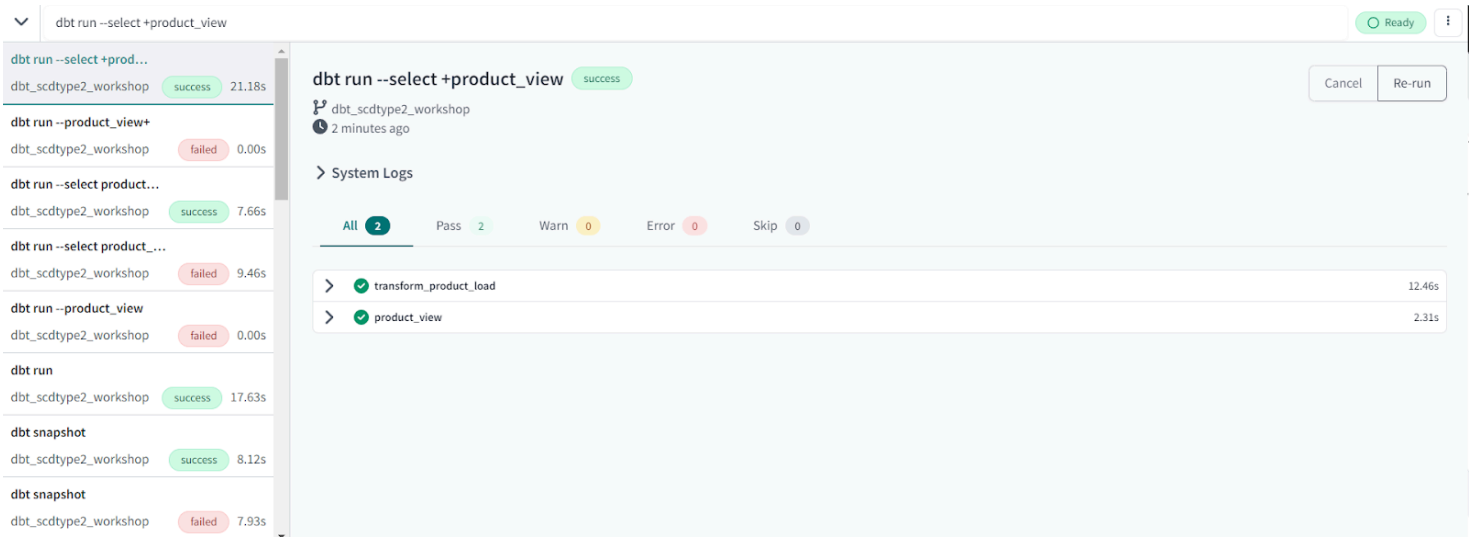
WITH view AS(
SELECT
    PRODUCT_ID AS PRODUCT_ID
    ,DBT_VALID_FROM AS VRSN_STRT_DTS
    ,COALESCE(DBT_VALID_TO , '9999-12-31 00:00:00.000') AS VRSN_END_DTS
    ,PRODUCT_NAME AS PRODUCT_NAME
    ,CATEGORY AS CATEGORY
    ,SELLING_PRICE AS SELLING_PRICE
    ,MODEL_NUMBER AS MODEL_NUMBER
    ,ABOUT_PRODUCT AS ABOUT_PRODUCT
    ,PRODUCT_SPECIFICATION AS PRODUCT_SPECIFICATION
    ,TECHNICAL_DETAILS AS TECHNICAL_DETAILS
    ,SHIPPING_WEIGHT AS SHIPPING_WEIGHT
    ,PRODUCT_DIMENSIONS AS PRODUCT_DIMENSIONS
    ,TIME_ZONE
    ,SOURCE_SYS_NAME
    ,INSTNC_ST_NM
```

```
    ,PROCESS_ID
    ,PROCESS_NAME
    ,INSERT_DTS
    ,UPDATE_DTS
FROM {{ref('product_snapshot')}}
)
SELECT *
FROM view
```



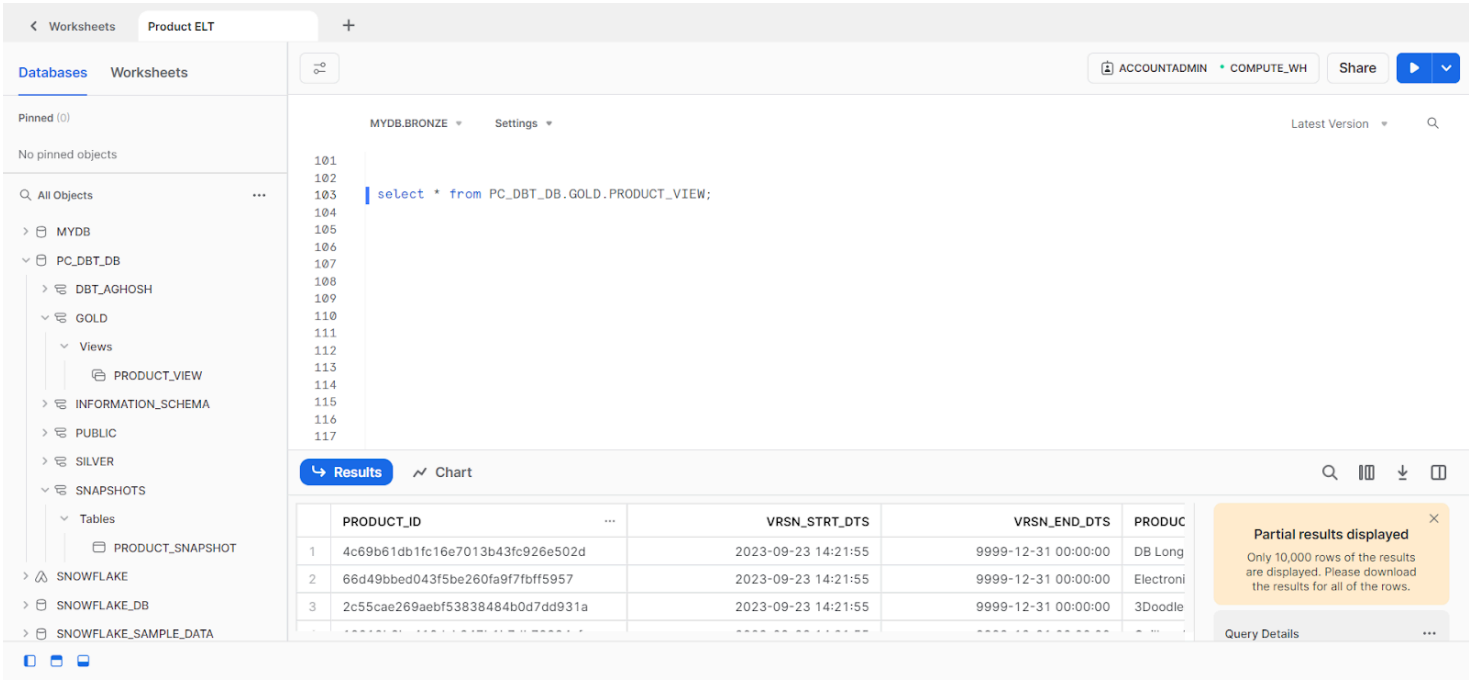
This will create a view on top of the snapshot table with all changes in data.

3. After all your models are saved and ready, run dbt run –select +product_view.



4. Run below select on your Snowflake worksheet to view the loaded data. The process is complete now.

select * from PC_DBT_DB.GOLD.PRODUCT_VIEW;



Successfully setup done of DBT Cloud with Source table created, Snapshots used to find change in data and observe data using view in Gold layer.

IMPORTANT: Legal Disclaimers

*Data Education Holdings LLC is not responsible for any costs incurred while completing this lesson. Data Education Holdings LLC is not part of Amazon.com, Inc. It is also not endorsed by Amazon, AWS, or any of the affiliated subsidiaries.

© Data Education Holdings LLC